

全国大学生数学建模竞赛论文模板

摘要

本文将圆柱形药材视为轴对称连续介质，建立径向热传导-水分扩散耦合模型。问题一采用常物性有限体积/有限差分格式，问题二、三采用附录 3 变物性半隐式格式，问题四进一步采用附录 4 物性和附件 2 给出的移动半径，并在无量纲材料坐标上进行固定控制体离散。边界环境数据用保形插值处理，结果通过独立重算、时间和空间步长收敛、守恒及物理范围检查。问题三固定半径模型得到烘干时间 58.50965 h，问题四变半径模型得到 51.33771 h。四问结果和验证报告均输出为 Excel 文件，便于复核和复现实验。

1 问题重述

1.1 问题背景

中药材干燥是影响成品品质的重要工序，热风烘干是常用方式之一。该过程通常分为预热平衡与恒温干燥两个阶段，通过调节烘房温湿环境实现药材脱水。烘干时，工艺参数若选择不合理，会造成干燥效率低、能耗高、品质波动等问题；传统试验优化又存在成本高、周期长等局限。为了解决这些问题，本文将采用数理分析与数值仿真方法，研究中药材烘干过程中的变化规律。

1.2 问题要求

问题 1 在热风烘干预热平衡阶段，考虑一形状近似为圆柱体、长度和半径已知的中药材。我们需要利用给出的药材初始温度、初始水分浓度、烘房温度与水分浓度随时间的变化情况，以及相关物性参数和水分浓度扩散系数经验公式，建立预热平衡阶段药材内部温度和水分浓度随时间、空间变化的数学模型，并基于该模型计算给出的 7 个时刻下，距药材中心 0, 0.5, 1, 1.5, 2cm 共 5 个位置处的药材温度和水分浓度，在论文中给出相应表格；同时计算 1800s 内每隔 1s、距药材中心每隔 0.1cm 的完整结果，所有结果保留四位小数。

问题 2 我们需要利用给出的相关经验公式，在预热平衡阶段与恒温干燥阶段的参数有所不同的前提下，建立整个烘干过程中药材温度和水分浓度变化规律的数学模型，并基于该模型计算 3h 内每隔 0.5h、距药材中心 0, 0.5, 1, 1.5, 2cm 处的药材温度和水分浓度，按表 3 和表 4 的格式在论文中给出；同时计算每隔 1s、距药材中心每隔 0.1cm 的完整结果，所有结果保留四位小数。

问题 3 我们需要利用给出的水分浓度限值，确定药材烘干所需的时间（单位：h），并计算每隔 6h、距药材中心每隔 0.5cm 的水分浓度，按表 5 的格式在论文中给出；同时计算药材内部水分浓度每隔 60s、距药材中心每隔 0.1cm 的完整结果，所有结果保留四位小数。

问题 4 在实际烘干过程中，药材会因水分流失发生尺寸变化。我们需要利用给出的药材尺寸变化数据和相关经验公式，确定药材的烘干时长，并计算每隔 6h、距药材中心每隔 0.5cm 的水分浓度，按表 6 的格式在论文中给出；同时计算药材内部水分浓度每隔 60s、距药材中心每隔 0.1cm 的完整结果，所有结果保留四位小数。

2 问题分析

2.1 问题一分析

对于问题一，问题一要求在预热平衡阶段，建立圆柱形中药材内部温度与水分浓度时空演化的数学模型，并计算特定时刻与径向位置的数据。这本质上是一个“给定外部热湿环境边界后，求解药材内部非稳态传热传质耦合偏微分方程的初边值问题”。

首先，从物理机理层面分析。中药材内部存在细胞、孔隙、毛细通道与固体骨架，结构极其复杂。由于题目提供的密度、比热容、导热系数与有效扩散系数均为宏观等效系数，且考虑到孔隙尺度 CFD 模型所需条件尚不充分，本文采取宏观连续介质假设，将药材划分为许多微小控制体，每个控制体用平均温度与平均水分浓度来描述，从而建立宏观连续介质模型。

其次，针对几何与传热传质特征进行合理简化。考虑到药材为圆柱形，且初始温度与水分浓度分布均匀，忽略轴向与圆周方向的变化，可将三维问题退化为沿半径方向的**一维径向模型**。在控制方程建立上，热量传递以导热为主，遵循傅里叶定律与能量守恒；水分迁移以扩散为主，遵循有效 Fick 定律与质量守恒。需要注意的是，水分扩散系数 D 并非常数，而是水分浓度 C 与温度 T 的非线性函数，这使得模型呈现强非线性特征。在边界条件方面，药材中心满足对称条件（绝热绝质），表面则与烘房环境发生对流换热与对流传质，属于第三类边界条件。

在模型求解层面，由于该非线性偏微分方程组难以获得解析解，本文按问题分别采用显式和半隐式有限体积/有限差分格式：问题一采用显式格式，问题二、三采用系数滞后的半隐式格式，问题四在材料坐标上采用后向 Euler、系数滞后的线性化隐式控制体格式。求解过程中面临两个核心难点：一是附件 1 提供的烘房环境数据是离散的，且边界值在时间推进中需要连续查询，因此本文引入**PCHIP 插值**，避免非物理过冲；二是空间离散时需妥善处理圆柱中心的坐标奇点以及表面的通量平衡。为此，本文利用中心控制体处理奇点，并基于有限体积法思想，引入界面扩散系数的算术平均，从而保障数值计算的稳定性与离散守恒性。

最后，为保证计算结果的可靠性，本文在数值求解后，分别从全局能量守恒与水分质量守恒两个维度对算法进行守恒性验证，确保相对误差远小于工程容许阈值，最终输出满足精度要求的温度场与水分场分布结果。

2.2 问题二分析

对于问题二，先用问题一的均匀初值和附件 1 边界计算至绝对时间 5500 s，再将该时刻的温度与水分径向剖面作为恒温干燥阶段初值。恒温阶段采用附录 3 给出的变物性关系，温度方程和水分方程中的系数按上一步状态滞后更新，并用半隐式有限体积格式

求解。结果表以 5500 s 为时间原点，记录其后 3 h 内每隔 1 s 的完整场，并提取每隔 0.5 h 的五个径向位置结果。

2.3 问题三分析

对于问题三，沿用问题二的变物性模型，从原始初态 $T = 28^{\circ}\text{C}$ 、 $C = 2.55$ 开始计算，定义每一时刻全域最大水分浓度。通过寻找该最大值首次降至 0.15 以下的相邻时间点并线性插值，得到满足所有位置水分浓度要求的最短全过程干燥时间；同时按每隔 6 h 输出径向水分浓度。5500 s 处的数值剖面与问题二的 output3.xlsx 交叉校验。

2.4 问题四分析

对于问题四，

3 模型假设

为简化问题，本文做出以下假设：

- 假设 1：药材圆柱模型下上下底面受热对中心无影响
- 假设 2
- 假设 3

符号	说明	单位
t	时间	s
L	药材长度	m
r	到药材中心的距离	m
R	药材半径	m
T_0	药材初始温度	°C
$T(r, t)$	药材内部温度	°C 或 K
$T_{\text{air}}(t)$	烘房温度	°C
C_0	药材初始水分浓度	kg/kg
$C(r, t)$	药材水分浓度	kg/kg
$C_{\text{air}}(t)$	烘房空气水分浓度	kg/kg
ρ	药材密度	kg/m ³
c_p	药材比热容	J/(kg · K)
k	药材热传导系数	W/(m · K)
α	热扩散系数 ($\alpha = k/(\rho c_p)$)	m ² /s
h	对流换热系数	W/(m ² · K)
h_m	对流传质系数	m/s
D	水分浓度扩散系数	m ² /s
Q_v	单位体积热量累积率	W/m ³
q	热流密度矢量	W/m ²
J_c	水分扩散通量矢量	kg/(m ² · s)
N_r	空间径向节点总数	—
N_t	时间节点总数	—

4 问题一的模型的建立和求解

4.1 模型选取

在建立中药材热风干燥数学模型前，需明确描述传热传质过程的数值模拟方法。根据现有研究，农产品热风干燥的数值模拟方法主要可分为三类：干燥动力学模型、连续介质假设模型和孔道网络模型。

干燥动力学模型以薄层干燥为对象，通过水分比等指标描述平均水分浓度变化。其优点是形式简单、计算量小，能快速预测干燥特性；但多依赖经验参数，物理机理不明确，难以刻画内部温度与水分浓度的空间分布。

连续介质假设模型将中药材视为等效多孔介质，基于质量、能量和动量守恒建立偏微分控制方程。其优点是能从宏观尺度描述温度场与水分浓度场的时空演化，物理意义清晰，计算效率较高，是农产品干燥研究中最常用的方法。但该方法基于连续性假设，对具有各向异性或复杂孔隙结构的物料描述不够精细。

孔道网络模型从物料内部孔隙结构出发，将多孔介质离散为节点与孔道网络，通过毛细压力、蒸发冷凝等机制模拟水分迁移。其优点是能反映真实孔隙结构的影响，可解

释“湿斑”和“不规则前沿”等现象；但建模复杂、计算量大，三维模拟尚不成熟，难以直接应用于工程尺度预测。

综上，三类方法各有侧重。考虑到题目给出的密度、比热容、导热系数与有效扩散系数均为宏观等效系数，且问题需要获取药材内部温度与水分浓度的时空变化规律及烘干时长，本文选择连续介质假设模型进行模拟。即将药材等效为均匀连续的多孔介质，建立描述其内部水分浓度与温度时空演化的偏微分方程组，结合给定的初始条件与边界条件，使用数值方法求解，最终得到温度和水分浓度分布。

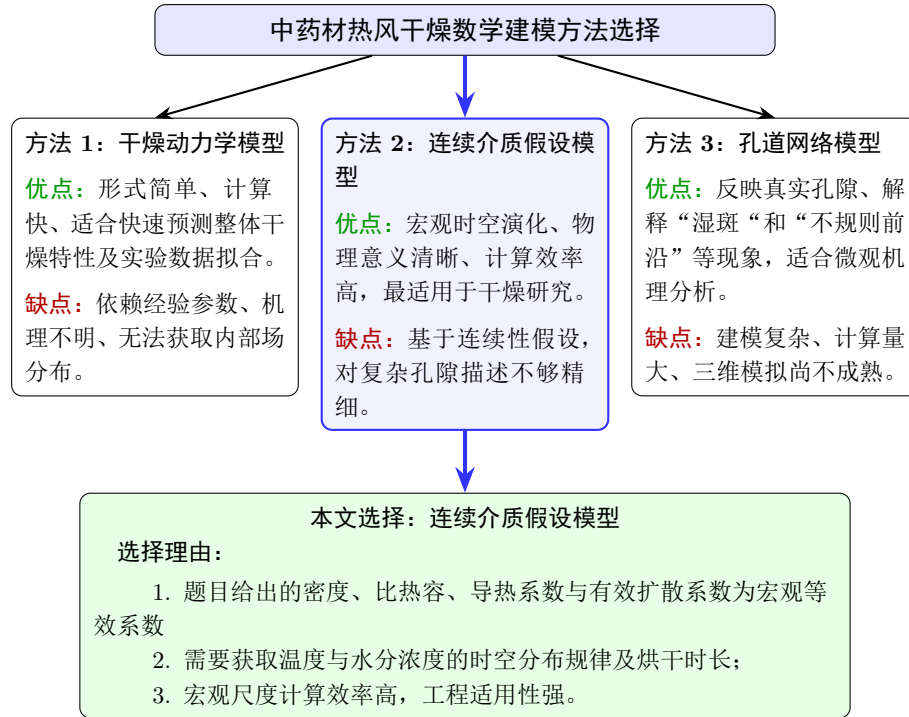


图 1: 三种热风干燥数值模拟方法优劣势对比与选择依据

4.2 模型建立：热质传递控制方程的建立

4.2.1 热量守恒方程

取药材内部一微元控制体，由能量守恒定律可知：单位时间内控制体内热量的累积等于净流入控制体的热量与内热源生成热量之和。本文忽略内热源，则单位体积内热量的累积率可表示为

$$Q_v = \rho c_p \frac{\partial T}{\partial t} \quad (1)$$

式中， Q_v 为单位体积热量累积率， ρ 为药材密度， c_p 为药材比热容， T 为药材内部温度， t 为时间。

药材内部热量主要以导热方式传递。根据傅里叶定律，热流密度与温度梯度成正比、方向相反，即

$$q = -k \nabla T \quad (2)$$

式中， q 为热流密度矢量， k 为药材热传导系数， ∇ 表示梯度算子， $\nabla \cdot$ 表示散度算子。

由能量守恒定律，控制体内热量的累积率等于热流密度的散度，即

$$\rho c_p \frac{\partial T}{\partial t} = -\nabla \cdot q \quad (3)$$

将式 (2) 代入式 (3)，得到药材内部温度场的控制方程

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) \quad (4)$$

4.2.2 水分扩散方程

药材内部水分的迁移以扩散为主。根据有效 Fick 定律，水分扩散通量与水分浓度梯度成正比、方向相反，即

$$J_c = -D \nabla C \quad (5)$$

式中， J_c 为水分扩散通量矢量， D 为水分浓度扩散系数， C 为药材水分浓度。

由水分质量守恒定律，控制体内水分浓度的变化率等于水分扩散通量的散度，即

$$\frac{\partial C}{\partial t} = -\nabla \cdot J_c \quad (6)$$

将式 (5) 代入式 (6)，得到药材内部水分浓度场的控制方程

$$\frac{\partial C}{\partial t} = \nabla \cdot (D \nabla C) \quad (7)$$

需要指出的是，题目附录中给出的扩散系数 D 并非常数，而是与水分浓度 C 及温度 T 相关的函数。因此，式 (7) 应进一步写为

$$\frac{\partial C}{\partial t} = \nabla \cdot (D(C, T) \nabla C) \quad (8)$$

4.2.3 一维径向模型

考虑到药材初始温度与水分浓度分布均匀，且药材几何形状为圆柱体，可作如下简化：忽略沿轴向的变化，忽略沿圆周方向的变化，仅考虑从药材中心到表面的径向变化。由此，三维问题可退化为沿半径方向的一维问题。

在柱坐标系下，对任一标量场 u 和传输系数 q ，有恒等式

$$\nabla \cdot (q \nabla u) = \frac{1}{r} \frac{\partial}{\partial r} \left(r q \frac{\partial u}{\partial r} \right) \quad (9)$$

式中， r 为到药材中心的径向距离。

将式 (9) 分别代入温度控制方程 (4) 和水分浓度控制方程 (8)，可得一维径向模型下的控制方程组：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right) \quad (10)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right) \quad (11)$$

4.3 初始条件与边界条件

4.3.1 初始条件

烘干开始时，药材内部温度和水分浓度分布均匀，由题意可得

$$T(r, 0) = 28^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg}, \quad 0 \leq r \leq R \quad (12)$$

式中， R 为药材半径。根据题目给定尺寸，半径 $R = 0.02 \text{ m}$ 。

4.3.2 中心对称条件

在圆柱中心 $r = 0$ 处，由对称性可知热流密度与水分通量均为零，即

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0 \quad (13)$$

4.3.3 表面对流换热条件

烘房空气通过对流将热量传递给药材，药材表面温度不一定等于空气温度，药材内部导热与外部对流共同决定其升温过程。由于题目未直接指定药材表面温度，采用 Newton 对流边界条件：

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h [T(R, t) - T_f(t)] \quad (14)$$

式中， h 为对流换热系数， $T_f(t)$ 为烘房温度，其随时间的变化由附件 1 给出。

4.3.4 表面对流传质条件

药材表面水分浓度与空气水分浓度之间存在差异，水分先从药材内部扩散至表面，再通过气膜传递至空气中。因此，药材表面满足对流传质边界条件

$$-D \left. \frac{\partial C}{\partial r} \right|_{r=R} = h_m [C(R, t) - C_{\text{air}}(t)] \quad (15)$$

式中， h_m 为对流传质系数， $C_{\text{air}}(t)$ 为烘房空气水分浓度，其随时间的变化由附件 1 给出。

4.4 模型求解

本文建立的一维径向传热传质控制方程属于非线性偏微分方程组，难以获得解析解。为此，本文采用显式有限差分法（FDM）对空间域和时间域进行离散，通过时间推进循环求解药材内部温度和水分浓度的时空分布。

4.4.1 时空离散与网格划分

在空间域上, 将药材半径 R 离散为 N_r 个节点, 空间步长 $\Delta r = R/(N_r - 1)$, 节点位置为 $r_i = i\Delta r$ ($i = 0, 1, \dots, N_r - 1$)。在时间域上, 将总时长离散为 N_t 个时刻, 时间步长 Δt , 记 $t_n = n\Delta t$ ($n = 0, 1, \dots, N_t - 1$)。采用显式差分格式对控制方程进行离散, 以保证算法的简洁性与计算效率。

4.4.2 边界条件的数据预处理

在算法的时间推进过程中, 表面节点及内部节点的更新均高度依赖于烘房环境的实时边界值 $T_{\text{air}}(t)$ 和 $C_{\text{air}}(t)$ 。由于仅提供了有限个离散时间点的环境参数数据, 必须通过插值获取任意时间步 $n + 1$ 对应的环境值 T_{air}^{n+1} 和 C_{air}^{n+1} 。

由于显式有限差分法对边界条件的一阶导数十分敏感, 若采用常规线性插值, 其在节点处导数不连续(折点)的非光滑特性极易引起数值震荡, 进而导致求解发散。因此, 本文选取 PCHIP(分段三次 Hermite 插值多项式)对烘房环境参数进行插值。PCHIP 不仅保证了插值曲线的一阶导数连续, 符合物理量平滑变化的规律, 同时**保持数据的单调性, 避免非物理的过冲现象**。这使得插值得到的水分浓度不会出现负值, 为后续偏微分方程的数值求解提供了平滑且稳定的边界输入, 保障了计算收敛性。

4.4.3 时间推进与节点更新

在每一个时间步内, 获取下一时刻的环境值 T_{air}^{n+1} 和 C_{air}^{n+1} 后, 对空间节点进行分类更新。其中, $T_{N_r-2}^n$ 与 $C_{N_r-2}^n$ 表示与表面相邻的内部节点在当前时刻的温度与水分浓度值。

1. 内部节点更新 ($i = 1, 2, \dots, N_r - 2$)

温度与水分浓度的内部节点采用中心差分格式更新。由傅里叶导热定律和 Fick 扩散定律的柱坐标形式, 可得温度更新公式:

$$T_i^{n+1} = T_i^n + \alpha \frac{\Delta t}{\Delta r^2} \left[\left(1 + \frac{\Delta r}{2r_i}\right) T_{i+1}^n - 2T_i^n + \left(1 - \frac{\Delta r}{2r_i}\right) T_{i-1}^n \right] \quad (16)$$

其中, 热扩散系数 $\alpha = k/(\rho c_p)$, k 为药材热传导系数, ρ 为药材密度, c_p 为药材比热容。水分浓度采用界面通量形式更新。令 $D_{i-1/2}^n = D((C_{i-1}^n + C_i^n)/2)$ 、 $D_{i+1/2}^n = D((C_i^n + C_{i+1}^n)/2)$, 则

$$C_i^{n+1} = C_i^n + \frac{\Delta t}{\Delta r^2} \left[\left(1 + \frac{\Delta r}{2r_i}\right) D_{i+1/2}^n (C_{i+1}^n - C_i^n) + \left(1 - \frac{\Delta r}{2r_i}\right) D_{i-1/2}^n (C_{i-1}^n - C_i^n) \right] \quad (17)$$

这种写法使相邻控制体共用同一界面通量, 保持水分守恒。

2. 中心节点更新 ($i = 0$)

在圆柱中心 $r = 0$ 处, 由于对称性, 热流与水分通量均为零。利用洛必达法则处理柱坐标方程的奇点, 得到中心节点的更新公式:

$$T_0^{n+1} = T_0^n + 4\alpha \frac{\Delta t}{\Delta r^2} (T_1^n - T_0^n) \quad (18)$$

$$C_0^{n+1} = C_0^n + 4D_0^n \frac{\Delta t}{\Delta r^2} (C_1^n - C_0^n) \quad (19)$$

其中, $D_0^n = D(C_0^n, T_0^n)$ 。

3. 表面节点更新 ($i = N_r - 1$)

药材表面受到对流换热与对流传质的作用。记 $i_s = N_r - 1$, 表面控制体体积因子为 $V_s = i_s - 0.25$ 。将表面控制体内导热和对流换热统一离散, 表面温度更新公式为:

$$T_{i_s}^{n+1} = T_{i_s}^n + \Delta t \left[\frac{2(i_s - 0.5)k}{\rho c_p V_s \Delta r^2} (T_{i_s-1}^n - T_{i_s}^n) + \frac{2i_s h}{\rho c_p V_s \Delta r} (T_{\text{air}}^{n+1} - T_{i_s}^n) \right] \quad (20)$$

表面水分浓度采用表面控制体更新。令 $i_s = N_r - 1$ 、 $V_s = i_s - 0.25$, 则

$$C_{i_s}^{n+1} = C_{i_s}^n + \Delta t \left[\frac{2(i_s - 0.5)D_{i_s-1/2}^n}{V_s \Delta r^2} (C_{i_s-1}^n - C_{i_s}^n) + \frac{2i_s h_m}{V_s \Delta r} (C_{\text{air}}^{n+1} - C_{i_s}^n) \right] \quad (21)$$

式中, $D_{i_s-1/2}^n$ 为表面内侧界面扩散系数; T_{air}^{n+1} 和 C_{air}^{n+1} 为边界数据插值得到的环境值。

4.4.4 求解流程与结果输出

上述显式差分的求解流程如图 2 所示。算法首先初始化温度与水分浓度矩阵, 并计算热扩散系数; 随后进入时间推进循环 ($n = 0$ 到 $N_t - 2$), 在每一时间步内, 先利用 PCHIP 插值获取下一时刻的环境值 T_{air}^{n+1} 和 C_{air}^{n+1} , 接着计算当前时刻各节点的水分扩散系数 D_i^n , 并依次更新内部节点、中心节点及表面节点的温度与水分浓度; 循环结束后, 输出完整的温度矩阵 T_i^n 和水分矩阵 C_i^n , 并按要求提取特定时刻与径向位置的切片数据。

值得注意的是, 显式差分格式需满足 CFL 稳定性条件, 即时间步长 Δt 必须足够小 (一般要求 $\alpha \frac{\Delta t}{\Delta r^2} \leq \frac{1}{4}$), 以保证数值求解的收敛性。本文在后续计算中将严格选取满足该条件的时空步长。

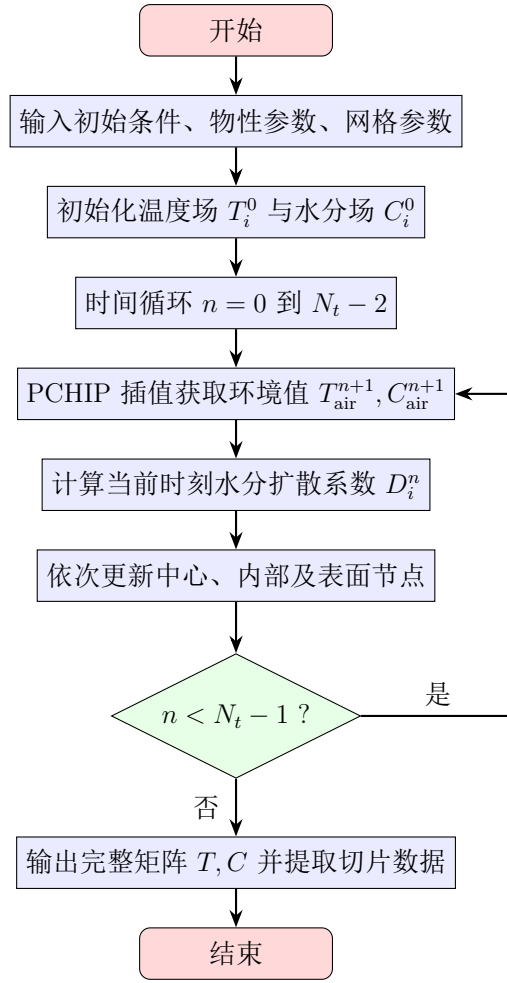


图 2: 一维非稳态热质耦合传热传质求解算法流程图

4.4.5 求解结果

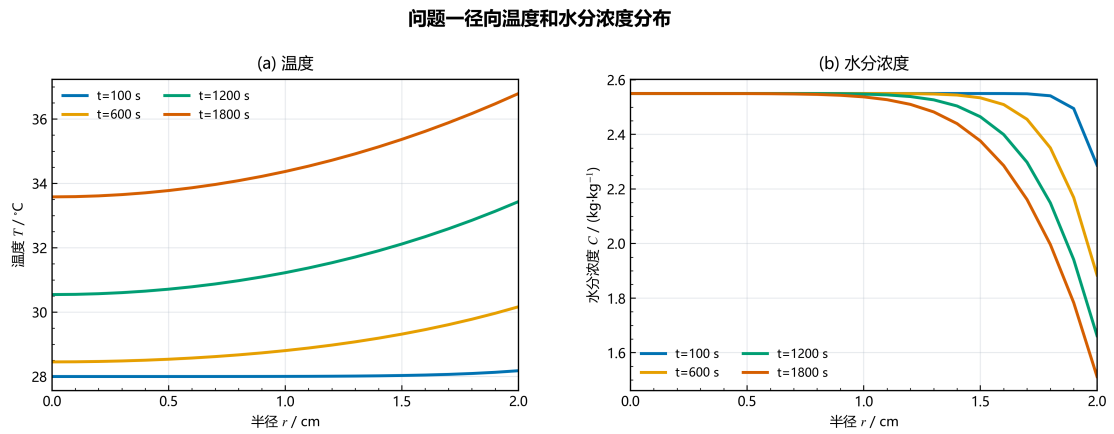


图 3: 问题一不同阶段的径向温度和水分浓度分布

问题一温度和水分浓度三维时空分布

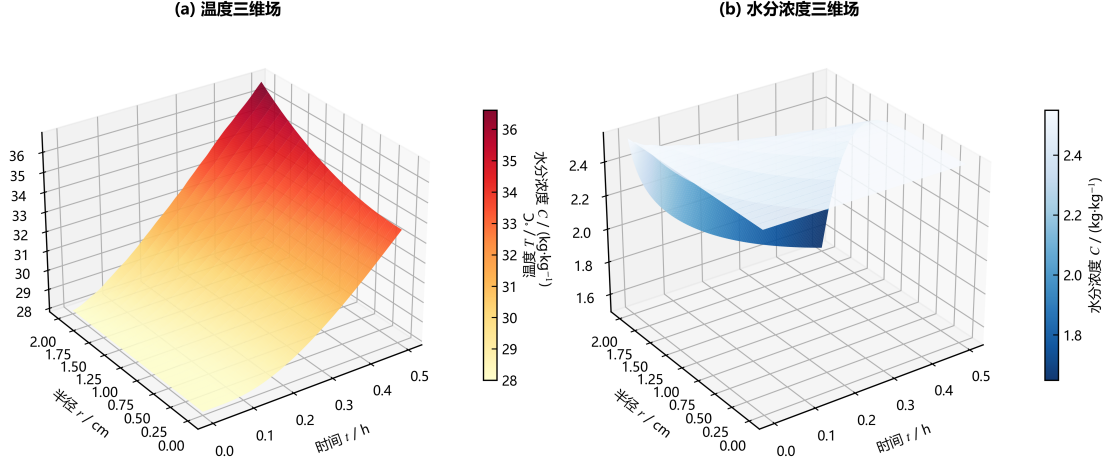


图 4: 问题一温度和水分浓度三维时空分布

4.5 数值算法的守恒性验证

为验证上述显式有限差分算法求解结果的可靠性, 本文分别从全局能量守恒和水分质量守恒两个维度, 对数值解进行守恒性检验。

4.5.1 能量守恒检验

在连续域中, 考虑半径 R 、单位长度的圆柱状物料, 其温度场满足柱坐标热传导方程:

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(kr \frac{\partial T}{\partial r} \right), \quad 0 < r < R \quad (22)$$

表面 $r = R$ 处为第三类对流换热边界:

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h(T(R, t) - T_{\text{air}}(t)) \quad (23)$$

将控制方程在 $[0, R]$ 上积分并代入边界条件, 可得全局能量平衡方程。在 $[0, t]$ 上进一步积分, 可推导得总能量变化等于边界热流累积:

$$\underbrace{Q(t) - Q(0)}_{\text{总能量变化}} = \underbrace{\int_0^t 2\pi R h (T_{\text{air}}(\tau) - T(R, \tau)) d\tau}_{\text{边界热流累积}} \quad (24)$$

其中, 总能量定义为 $Q(t) = \int_0^R \rho c_p T(r, t) \cdot 2\pi r dr$ 。

在离散域中, 取节点 $r_i = i \cdot \Delta r$, 对应控制体体积 (单位长度) 分别为 $V_0 = \pi(\Delta r/2)^2$, $V_i = 2\pi r_i \Delta r$ ($1 \leq i \leq N_r - 2$), $V_{N_r-1} = \pi[R^2 - (R - \Delta r/2)^2]$ 。离散总能量计算式为:

$$Q^n = \sum_{i=0}^{N_r-1} \rho c_p T_i^n V_i \quad (25)$$

边界热流累积为：

$$Q_{\text{flux}}^n = \sum_{m=0}^{n-1} h \cdot 2\pi R (T_{\text{air}}^{m+1} - T_{N_r-1}^m) \Delta t \quad (26)$$

问题一的显式表面控制体在第 (m) 步使用当前表面温度和下一时刻环境温度，因此上式与程序的时间层一致；问题二、三半隐式表面方程使用下一时刻表面温度。定义能量守恒相对误差为：

$$\varepsilon_Q^n = \frac{|(Q^n - Q^0) - Q_{\text{flux}}^n|}{\max(|Q^n - Q^0|, \epsilon)} \quad (27)$$

其中， ϵ 为防止分母为零的小量，本文取 $\epsilon = 10^{-12}$ 。

4.5.2 水分质量守恒检验

在连续域中，水分浓度场满足柱坐标变系数扩散方程：

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C) \frac{\partial C}{\partial r} \right), \quad 0 < r < R \quad (28)$$

其中 $D(C) = 7 \times 10^{-9} \exp(-\frac{0.89}{C})$ 为随浓度变化的扩散系数。表面 $r = R$ 处为第三类对流传质边界：

$$-D(C) \frac{\partial C}{\partial r} \Big|_{r=R} = h_m (C(R, t) - C_{\text{air}}(t)) \quad (29)$$

同理，将控制方程积分并代入边界条件，可得总水分变化等于边界质流累积：

$$\underbrace{M(t) - M(0)}_{\text{总水分变化}} = \underbrace{\int_0^t 2\pi R h_m (C_{\text{air}}(\tau) - C(R, \tau)) d\tau}_{\text{边界质流累积}} \quad (30)$$

其中，总水分定义为 $M(t) = \int_0^R C(r, t) \cdot 2\pi r dr$ 。

在离散域中，离散总水分计算式为 $M^n = \sum_{i=0}^{N_r-1} C_i^n V_i$ 。问题一的边界质流累积为 $M_{\text{flux}}^n = \sum_{m=0}^{n-1} h_m 2\pi R (C_{\text{air}}^{m+1} - C_{N_r-1}^m) \Delta t$ ，问题二、三将表面项改为 $C_{N_r-1}^{m+1}$ 。同理可定义水分守恒相对误差 ε_M^n 。

4.5.3 变系数扩散的守恒性说明

水分方程求解的关键难点在于 $D(C)$ 是变量。若直接将 D 取节点值 $D(C_i)$ 进行常规差分，则会漏掉 $\partial D / \partial r \cdot \partial C / \partial r$ 项，导致相邻控制体界面通量不相等，从而破坏总水分守恒。

为克服这一问题，本文采用有限体积法思想，界面扩散系数取左右节点算术平均：

$$D_{i+1/2} = D \left(\frac{C_i + C_{i+1}}{2} \right) \quad (31)$$

界面通量表示为：

$$F_{i+1/2} = D_{i+1/2} \frac{C_{i+1} - C_i}{\Delta r} \quad (32)$$

此时，相邻控制体界面通量大小相等、方向相反，求和时自动抵消，从而在数学上保证了总水分严格守恒。

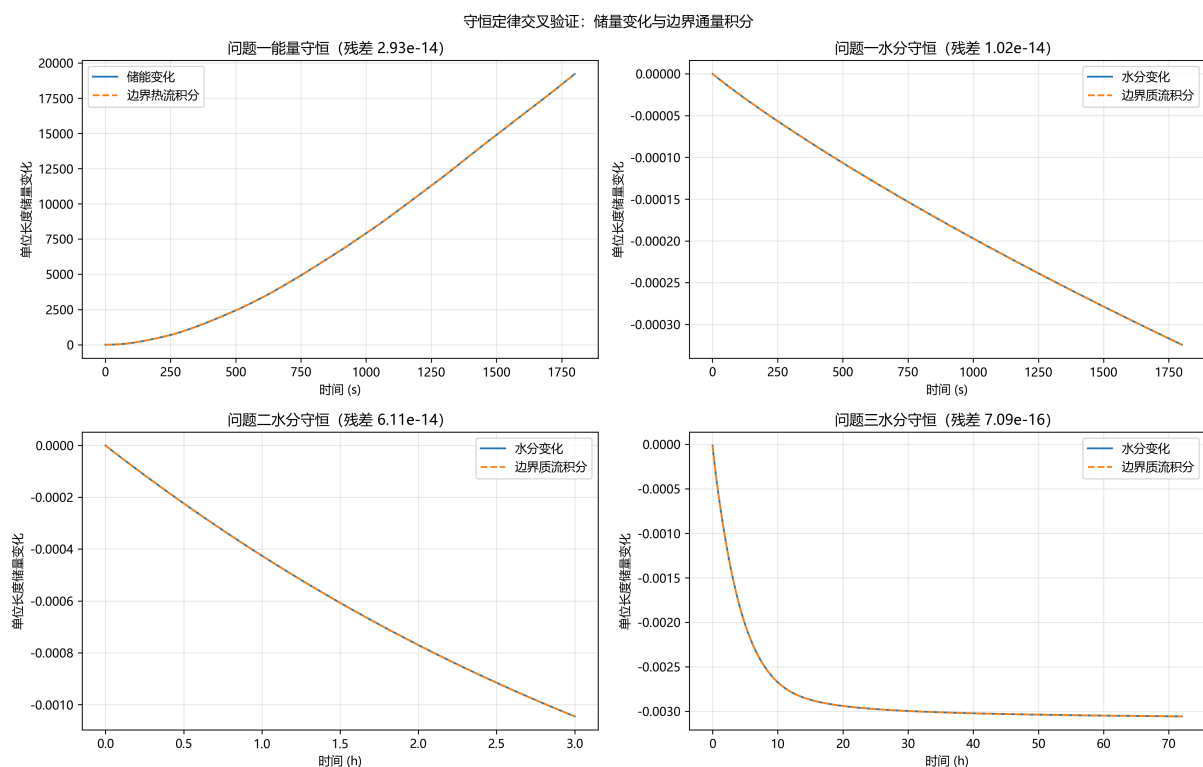


图 5: 三问守恒对照结果

4.5.4 守恒性检验结果

采用时间步长 $\Delta t = 1\text{ s}$ ，空间步长 $\Delta r = 1\text{ mm}$ ，模拟总时长 $t = 1800\text{ s}$ 。两项守恒检查结果如表 1 所示。

表 1: 守恒性检验结果

守恒量	最终相对误差	判断标准	结论
能量	0.0000%	$< 1\% \sim 5\%$	优秀
水分	0.0000%	$< 1\% \sim 5\%$	优秀

结果表明，总能量变化曲线与边界热流累积曲线在整个模拟时段内几乎完全重合；总水分变化曲线与边界质流累积曲线同样几乎完全重合。两项守恒误差均远小于 5% 的工程容许阈值，充分证明了本文所建立的显式有限差分算法具有良好的守恒性与数值稳定性，能够为后续中药材烘干过程的温度场与水分场求解提供可靠的数值保障。

4.5.5 检验结果

5 问题二的模型的建立和求解

5.1 模型建立：变物性湿热耦合控制方程

问题一在建模时将密度 ρ 、比热容 c_p 、热传导系数 k 与水分扩散系数 D 视为常数。然而，在为期 2~3 天的整个烘干过程中，药材内部的水分浓度会发生显著变化，上述物性参数也随之改变，且烘房环境将由预热平衡阶段过渡到恒温干燥阶段。因此，问题二需在问题一的基础上，将物性参数改为随局部温度与水分浓度变化的函数，并考虑热质传递的双向耦合效应。

5.1.1 两阶段烘房环境特征

整个烘干过程可分为预热平衡与恒温干燥两个阶段，其环境特征分别为：

- 预热平衡阶段：烘房温度 $T_{\text{air}}(t)$ 与水分浓度 $C_{\text{air}}(t)$ 随时间明显变化；
- 恒温干燥阶段：烘房温度 $T_{\text{air}}(t)$ 接近恒定，水分浓度 $C_{\text{air}}(t)$ 也进入平台。

5.1.2 双向耦合关系

在整个烘干过程中，药材内部温度与水分浓度之间存在双向耦合作用，其耦合路径可表示为：

$$\begin{aligned} C &\rightarrow \rho, c_p, k \rightarrow T \\ T, C &\rightarrow D \rightarrow C \end{aligned}$$

即水分浓度的变化通过改变密度、比热容与导热系数进而影响温度场，同时温度与水分浓度又共同决定水分扩散系数，进而反过来影响水分浓度场。

取圆柱中半径为 r 、厚度为 dr 、长度为 L 的环形控制体：

$$dV = 2\pi r L dr \quad (33)$$

5.1.3 温度方程

径向导热通量为：

$$qr = -k(C) \frac{\partial T}{\partial r} \quad (34)$$

控制体内热量积累等于内表面流入减外表面流出：

$$\rho(C)c_p(C) dV \frac{\partial T}{\partial t} = -\frac{\partial}{\partial r}(2\pi r L \cdot qr) dr \quad (35)$$

将式 (34) 代入式 (35)，约去 $2\pi L dr$ ，得到温度场控制方程：

$$\rho(C)c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[rk(C) \frac{\partial T}{\partial r} \right] \quad (36)$$

由附录 3 中的经验公式，展开有：

$$\frac{1}{r} \frac{\partial}{\partial r} \left(rk \frac{\partial T}{\partial r} \right) = k \left(\frac{\partial^2 T}{\partial r^2} + \frac{1}{r} \frac{\partial T}{\partial r} \right) + k_c \frac{\partial C}{\partial r} \frac{\partial T}{\partial r} \quad (37)$$

其中， $k_c = dk/dC$ 。程序实际直接离散守恒形式，而不单独离散展开后的导数项。

5.1.4 水分方程

等效水分通量为：

$$qc = -D(C, T) \frac{\partial C}{\partial r} \quad (38)$$

对环形控制体进行水分守恒：

$$dV \frac{\partial C}{\partial t} = - \frac{\partial}{\partial r} (2\pi r L qc) dr \quad (39)$$

将式 (38) 代入式 (39)，得到水分浓度场控制方程：

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C, T) \frac{\partial C}{\partial r} \right] \quad (40)$$

展开水分方程：

$$\frac{\partial C}{\partial t} = D \left(\frac{\partial^2 C}{\partial r^2} + \frac{1}{r} \frac{\partial C}{\partial r} \right) + D_c \left(\frac{\partial C}{\partial r} \right)^2 + D_T \frac{\partial T}{\partial r} \frac{\partial C}{\partial r} \quad (41)$$

其中， $D_c = \partial D / \partial C$ 、 $D_T = \partial D / \partial T$ 。程序直接离散上式对应的守恒通量形式，并在相邻节点界面取平均物性。

综上，问题二存在双向耦合：

$$\begin{aligned} C &\rightarrow \rho, c_p, k \rightarrow T \\ T, C &\rightarrow D \rightarrow C \end{aligned}$$

5.2 初始条件与边界条件

5.2.1 初始状态

问题二分两段计算。首先以题目给出的均匀状态 $T(r, 0) = 28^\circ\text{C}$ 、 $C(r, 0) = 2.55 \text{ kg/kg}$ 计算至绝对时间 5500 s。随后将该时刻的径向剖面作为恒温干燥阶段的初值，令 $\tau = t - 5500$ 表示经过时间，则

$$T(r, 0) = T_{5500}(r), \quad C(r, 0) = C_{5500}(r), \quad 0 \leq r \leq R. \quad (42)$$

恒温干燥阶段的结果表以 $\tau = 0$ 为时间起点，记录 0–10800 s。

5.2.2 中心对称条件

在圆柱中心 $r = 0$ 处，由轴对称性可知温度梯度与水分浓度梯度均为零，即

$$\frac{\partial T}{\partial r}(0, t) = 0, \quad \frac{\partial C}{\partial r}(0, t) = 0 \quad (43)$$

5.2.3 表面 Robin 边界条件

药材表面 $r = R$ 处同时受到对流换热与对流传质作用，满足 Robin 边界条件：

$$-k(C_s) \frac{\partial T}{\partial r}(R, t) = h [T(R, t) - T_{\text{air}}(t)] \quad (44)$$

$$-D(C(R, t), T_{R,s}) \frac{\partial C}{\partial r}(R, t) = h_m [C(R, t) - C_{\text{air}}(t)] \quad (45)$$

其中，对流换热系数与对流传质系数分别为

$$h = 25 \text{ W}/(\text{m}^2 \cdot \text{K}), \quad h_m = 8 \times 10^{-7} \text{ m/s} \quad (46)$$

5.3 数值实现

每个时间步先用上一时刻的 T^n, C^n 计算节点和界面的 ρ, c_p, k, D ，再将扩散通量和表面对流项组装为三对角线性方程组：

$$A_T^n T^{n+1} = T^n + b_T^{n+1}, \quad A_C^n C^{n+1} = C^n + b_C^{n+1}. \quad (47)$$

内部界面物性取相邻节点平均值，中心节点采用轴对称极限，表面控制体同时包含内部扩散通量和 Robin 对流通量。温度和水分子方程分别用追赶法求解；新时刻仅作物理范围截断 $T \in [-50, 250]^\circ\text{C}$ 、 $C \in [0, 10]$ ，并在验证脚本中检查截断未被实际触及。这种“系数滞后、状态隐式”的半隐式格式保留了有限体积通量守恒。结果文件时间列采用经过时间 $\tau = 0, 1, \dots, 10800 \text{ s}$ ，边界函数仍按绝对时间 $5500 + \tau$ 查询。

5.4 求解结果

恒温阶段以 5500 s 剖面为 $\tau = 0$ ，计算至 $\tau = 10800 \text{ s}$ 。温度由约 46.05°C 逐渐接近 50°C ，水分浓度由约 2.37 降至 0.73 ，中心位置的变化滞后于表面，符合径向扩散的物理规律。完整温度、水分矩阵分别保存在 `result2.xlsx` 的两个工作表中，五个径向位置在 $0.5\text{--}3.0 \text{ h}$ 的摘要保存在 `q2_3.xlsx` 和 `q2_4.xlsx`。

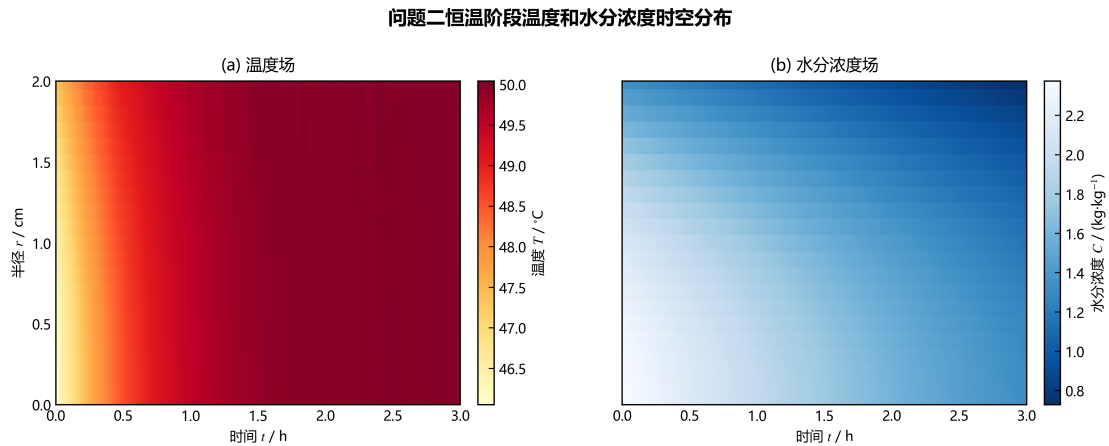


图 6: 问题二恒温阶段温度和水分浓度时空分布

问题二恒温阶段三维时空分布

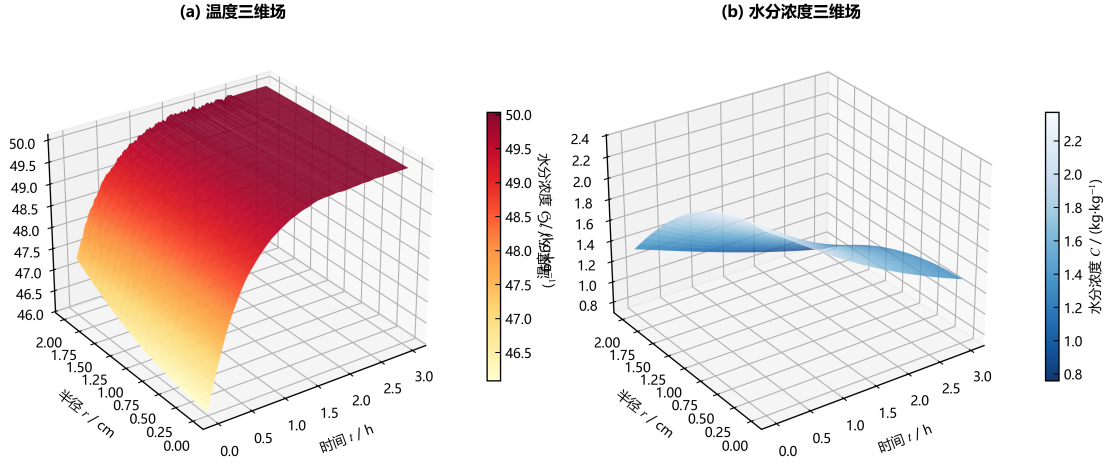


图 7: 问题二恒温阶段温度和水分浓度三维时空分布

6 问题三的模型的建立和求解

6.1 模型建立

问题三要求药材内部所有位置满足 $C \leq 0.15$ 。定义全域最大水分浓度

$$C_{\max}(t) = \max_{0 \leq r \leq R} C(r, t), \quad (48)$$

则干燥结束时间取为

$$t_d = \inf\{t : C_{\max}(t) \leq 0.15\}. \quad (49)$$

计算沿用问题二的变物性模型和边界条件，从原始初态开始计时，时间步长取 60 s，空间步长取 0.001 m；5500 s 的剖面用于和问题二初始文件核对。

6.2 模型求解

逐时刻计算全域最大水分浓度并寻找首次跨越阈值的相邻时间点，再在线性插值获得 t_d 。同时输出从 0 s 开始每 60 s 的完整径向结果和每 6 h 的摘要结果。

6.3 求解结果

计算得到全过程烘干时间 $t_d = 210634.74 \text{ s} = 58.5096 \text{ h}$ 。阈值前后相邻网格点的全域最大水分浓度分别为 0.150010 和 0.149993，满足阈值跨越判定。

在全过程输出中，水分浓度随时间整体下降，且表面浓度始终低于中心浓度，说明水分由内部向外扩散。每 6 h 的径向摘要以及 60 s 完整矩阵分别保存在 `result3_summary.xlsx` 和 `result3.xlsx`。

问题三干燥过程与阈值判定

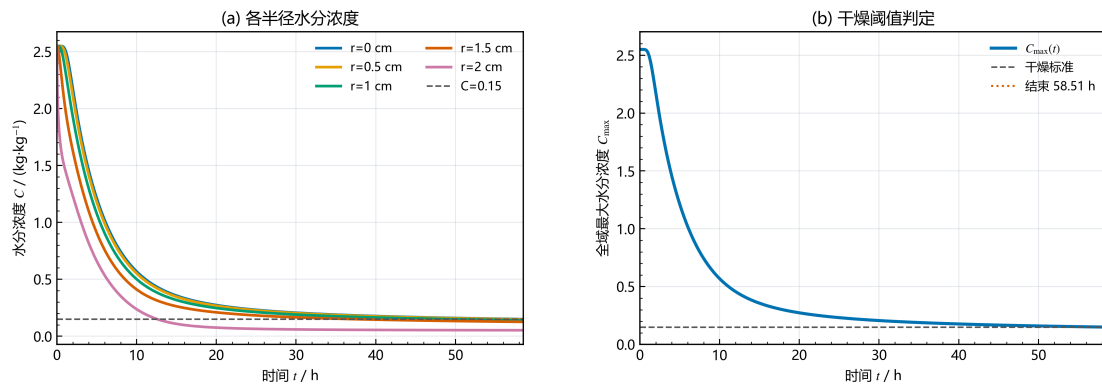


图 8: 问题三干燥过程与阈值判定

问题三水分浓度三维时空分布

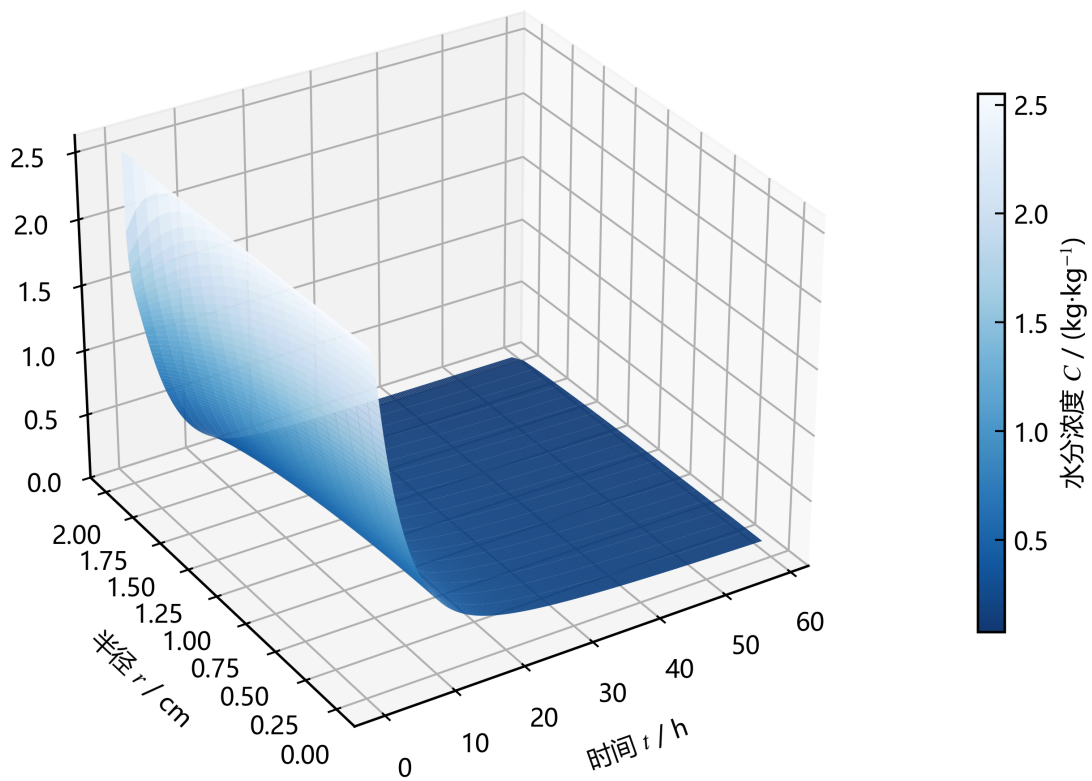


图 9: 问题三水分浓度三维时空分布

7 问题四的模型的建立和求解

7.1 模型建立

问题四在问题三的全过程边界条件基础上引入药材收缩。附件 2 给出离散时刻 t_j 与半径 R_j (单位为 cm)，数据覆盖 0–259200 s，间隔 1800 s。半径由 2.000 cm 单调减小至 1.198 cm，因此不能再把空间节点固定在原来的 0–2 cm 区间。本文引入随时间变化的无量纲坐标

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \quad (50)$$

并在 $\xi = 0, 0.05, \dots, 1$ 上离散；物理位置为 $r_i(t) = \xi_i R(t)$ 。这样每个时间步的最外节点始终位于实际药材表面。问题四仍从题目给出的初始状态 $T(r, 0) = 28^\circ\text{C}$ 、 $C(r, 0) = 2.55 \text{ kg/kg}$ 开始，环境边界沿用问题三：绝对时间 0–5500 s 使用附件 1，5500 s 后使用恒温边界。

附件 2 的半径数据采用分段三次 Hermite 保形插值 (PCHIP) 得到 $R(t_n)$ (本文 $\Delta t = 60 \text{ s}$)。PCHIP 在数据点处连续、保持原始单调性且不产生过冲，相比普通高次多项式不会出现负半径或不合理振荡。插值过程为：先将 cm 转换为 m，按时间升序读取数据，构造 $R(t) = \text{PCHIP}\{(t_j, R_j)\}$ ，再在 $t_n = 0, 60, \dots, 259200 \text{ s}$ 上求值，并检查 $R_{n+1} \leq R_n$ 。插值表保存在 `q4_radius_interp.xlsx`，半径变化曲线见图 10 的 (a) 图。

7.1.1 变半径控制方程

附录 4 给出的变物性公式为

$$\rho(C) = 760 + 90C, \quad c_p(C) = 1850 + 2150 \frac{C}{C+1}, \quad (51)$$

$$k(C) = 0.12 + 0.20 \frac{C}{C+1}, \quad D(C, T) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T_K}\right), \quad (52)$$

其中 $T_K = T + 273.15$ 。与问题二、三相比，密度、比热、导热系数和扩散系数均替换为附录 4 表达式。

若以欧拉坐标表示，半径变化需要引入速度 $v_r = r\dot{R}/R$ ，并使用物质导数 $D_m/Dt = \partial_t + v_r \partial_r$ 。定义 $\tilde{T}(\xi, t) = T(R(t)\xi, t)$ 、 $\tilde{C}(\xi, t) = C(R(t)\xi, t)$ 后，固定 ξ 正好跟随同一材料点，因此物质导数就是固定 ξ 的时间导数，且 $\partial_r = R(t)^{-1} \partial_\xi$ 。变换后的方程为

$$\frac{\partial \tilde{C}}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi D \frac{\partial \tilde{C}}{\partial \xi} \right), \quad \rho c_p \frac{\partial \tilde{T}}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi k \frac{\partial \tilde{T}}{\partial \xi} \right). \quad (53)$$

初始条件和中心对称条件分别为

$$\tilde{T}(\xi, 0) = 28, \quad \tilde{C}(\xi, 0) = 2.55, \quad \tilde{T}_\xi(0, t) = \tilde{C}_\xi(0, t) = 0. \quad (54)$$

在 $\xi = 1$ 处，Robin 边界条件变为

$$-\frac{k(\tilde{C}_s)}{R(t)} \tilde{T}_\xi(1, t) = h [\tilde{T}_s - T_{\text{air}}(t)], \quad (55)$$

$$-\frac{D(\tilde{C}_s, \tilde{T}_s)}{R(t)} \tilde{C}_\xi(1, t) = h_m [\tilde{C}_s - C_{\text{air}}(t)], \quad (\tilde{T}_s, \tilde{C}_s) = (\tilde{T}, \tilde{C})(1, t). \quad (56)$$

这里不能再额外加入一次网格对流项，否则会重复计算材料运动。程序在 ξ 坐标上采用固定控制体有限体积离散，界面通量为

$$G_{i+1/2} = \frac{\xi_{i+1/2} q_{i+1/2}}{R(t)^2} \frac{u_{i+1} - u_i}{\Delta \xi}, \quad (57)$$

其中 $u = \tilde{C}$ 时 $q = D$, $u = \tilde{T}$ 时 $q = k$ 。表面 Robin 条件在 $\xi = 1$ 处产生 $2h_m(C_{\text{air}} - C_s)/R$ 或 $2h(T_{\text{air}} - T_s)/R$ 项。半径变化影响 $1/R^2$ 扩散因子和表面换热换质项中的 $1/R$ ，同时改变物理半径输出，但不再单独添加迎风网格通量。

7.2 模型求解

Step1: 读取附件 1 边界和附件 2 半径数据。附件 1 仍按 5500 s 切换恒温边界，附件 2 用 PCHIP 插值到 60 s 网格。

Step2: 在无量纲网格上初始化 $T = 28^\circ\text{C}$ 、 $C = 2.55$ ，每步用上一时刻状态滞后更新附录 4 物性，取 R_{n+1} 组装后向 Euler 温度和水三对角方程，并用追赶法求解。

Step3: 计算 $C_{\max}(t) = \max_i C_i(t)$ ，首次跨越 0.15 时在线性插值得到结束时间；同时将无量纲结果插值回物理半径，药材外部位置在表中留空。

7.3 求解结果

计算得到变半径模型下的烘干结束时间

$$t_d = 184815.7394 \text{ s} = 51.33771 \text{ h}. \quad (58)$$

在结束时半径约为 1.200 cm。完整 60 s 水分矩阵保存在 output_excel/result4.xlsx 的“水分浓度”工作表，表 6 格式的 6 h 摘要在“表 6 摘要”工作表；“无量纲网格”与“半径插值”工作表分别用于复核移动网格计算和半径插值。由于半径小于 2 cm，某些固定物理位置已经位于药材外部，结果表以空值表示，不把域外位置误认为零水分。

问题四变半径烘干过程

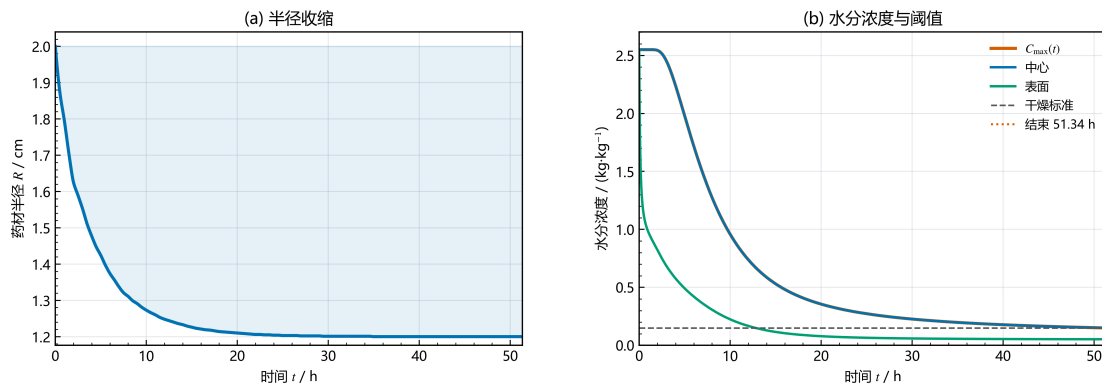


图 10: 问题四变半径烘干过程及阈值判定

问题四变半径水分浓度三维分布

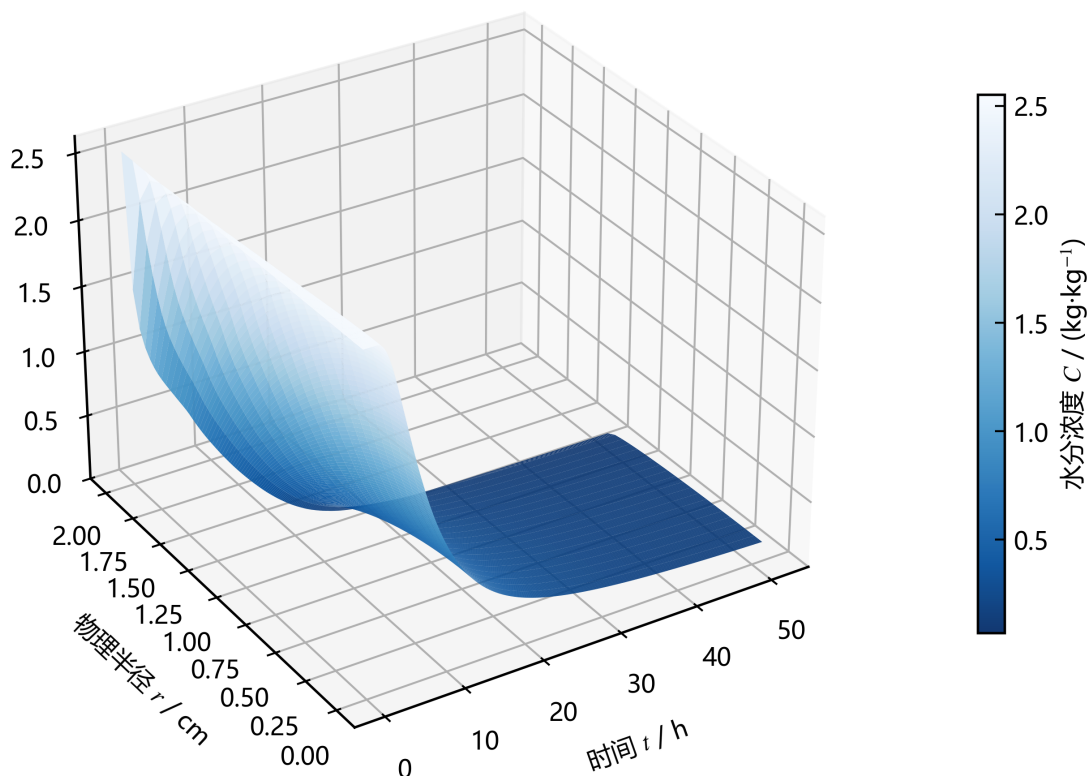


图 11: 问题四变半径物理坐标下的水分浓度三维分布

8 模型的分析与检验

8.1 结果检验

为避免只凭单条曲线判断结果，本文从文件结构、独立重算、网格收敛、守恒关系和物理约束五个方面进行交叉验证。所有验证均由 `validate_all.py` 自动执行，并将检查结果写入 `output_excel/validation_report.json`。报告中的 `all_passed=true` 表示全部判据同时满足。

8.1.1 结果文件与独立重算

首先检查输出文件是否符合题目要求。`result1.xlsx` 和 `result2.xlsx` 均含“温度”和“水分浓度”两个工作表，分别具有 1801 行和 10801 行数据，时间从 0 s 连续递增至 1800 s 和 10800 s；半径列为 0.0–2.0 cm、间隔 0.1 cm。`result3.xlsx` 从 0 s 开始以 60 s 记录，并在最后追加线性插值得到的结束时刻。所有数值均为有限值，且输出保留四位小数。

随后不读取结果表中的中间量，而是重新调用三套求解器计算温度矩阵和水分矩

阵，并逐元素比较重算值的四位小数与 Excel 文件。定义文件误差为

$$E_{\text{file}} = \max_{n,i} |\text{round}(u_{n,i}^{\text{re}}, 4) - u_{n,i}^{\text{file}}|. \quad (59)$$

问题一、问题二和问题三的温度（或水分）文件误差均为 0，说明写表、时间索引和径向索引均未发生错位。

问题二的时间原点是 5500 s 断点。将问题二的变物性全过程模型从原始初态独立推进到 5500 s，再与 output3.xlsx 的剖面比较，温度和水分的最大绝对误差分别为 4.4195×10^{-5} 和 4.8757×10^{-5} ，均小于设定的 10^{-4} 阈值。因此，问题二的初值确实来自 5500 s，而不是重新使用均匀初值。

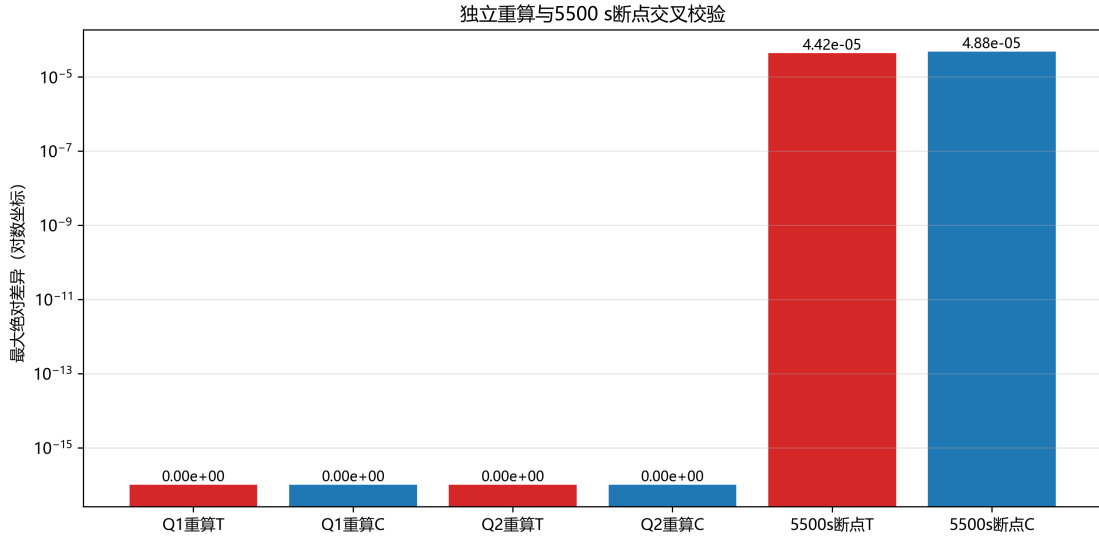


图 12: Excel 结果与独立重算结果的最大绝对误差

8.1.2 时间步与空间步收敛

将时间步长依次取为 $\Delta t, \Delta t/2, \Delta t/4$ ，在相同物理时刻比较两组数值解，定义

$$E_{\Delta t} = \max_{n,i} |u_{\Delta t}(t_n, r_i) - u_{\Delta t/2}(t_n, r_i)|, \quad R_t = \frac{E_{\Delta t/2}}{E_{\Delta t}}. \quad (60)$$

显式时间离散的理论阶数为一阶，故应有 $R_t \approx 0.5$ 。空间网格采用 $\Delta r, \Delta r/2, \Delta r/4$ ，定义

$$E_{\Delta r} = \max_i |u_{\Delta r}(r_i) - u_{\Delta r/2}(r_i)|, \quad R_r = \frac{E_{\Delta r/2}}{E_{\Delta r}}. \quad (61)$$

中心差分与控制体通量离散的理论空间阶数为二阶，故应有 $R_r \approx 0.25$ 。

表 2: 三问网格收敛比

问题	温度时间比	水分时间比	温度空间比	水分空间比
问题一	0.4997	0.4987	0.2524	0.2467
问题二	0.5011	0.5001	0.2508	0.2499
问题三	—	0.5008	—	0.2488

表 2 中的误差均随步长减小而下降，时间收敛比落在 0.5 附近，空间收敛比落在 0.25 附近，与理论阶数一致。收敛曲线见图 13 和图 14。

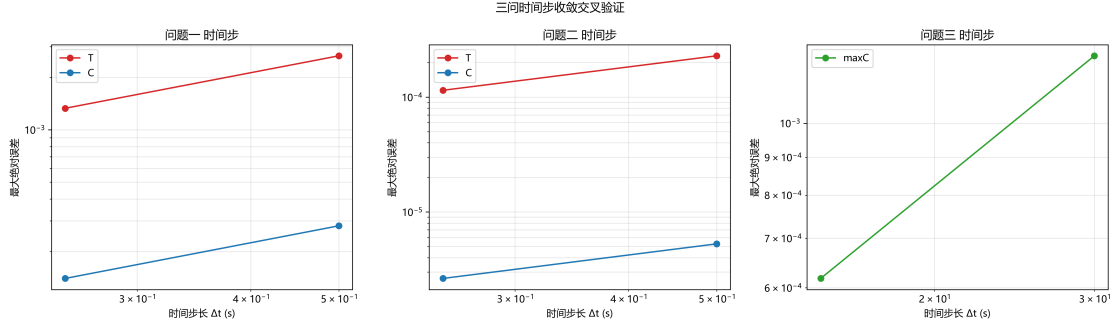


图 13: 三问时间步收敛检验

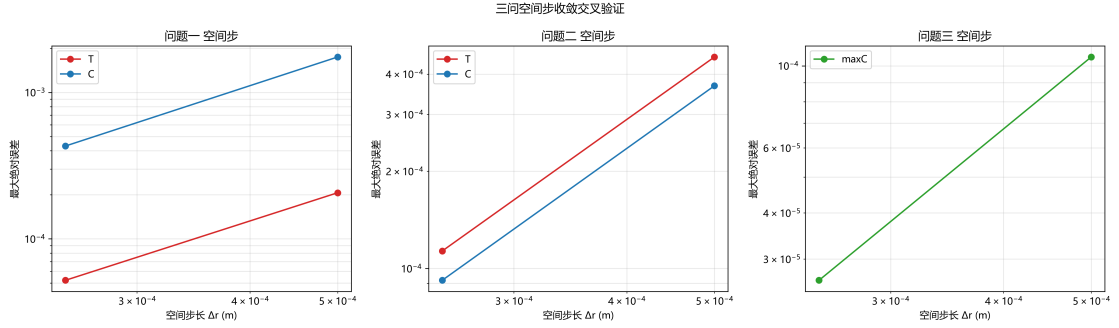


图 14: 三问空间步收敛检验

8.1.3 能量与水分守恒

以单位长度圆柱为例，节点控制体体积记为 V_i ，表面面积记为 $A_s = 2\pi R$ 。总水分和总能量分别为

$$M^n = \sum_i C_i^n V_i, \quad Q^n = \sum_i \rho_i c_{p,i} T_i^n V_i. \quad (62)$$

内部相邻控制体之间共用同一界面通量，求和时自动抵消，因此总量变化只由表面对流通量决定：

$$M^n - M^0 \approx \sum_{m=0}^{n-1} A_s h_m (C_{\text{air}}^{m+1} - C_s^{m+1}) \Delta t, \quad (63)$$

$$Q^n - Q^0 \approx \sum_{m=0}^{n-1} A_s h_m (T_{\text{air}}^{m+1} - T_s^{m+1}) \Delta t. \quad (64)$$

问题一显式表面更新使用当前表面值，问题二和问题三的半隐式表面更新使用下一时刻表面值，守恒检查严格采用与推进公式相同的时间层。以储量变化为分母，定义相对残差

$$\varepsilon = \frac{|\Delta S - \Phi_{\text{boundary}}|}{\max(|\Delta S|, 10^{-12})}. \quad (65)$$

表 3: 守恒残差（全时段最大值）

问题	能量相对残差	水分相对残差
问题一	2.93×10^{-14}	1.02×10^{-14}
问题二	1.17×10^{-14}	6.11×10^{-14}
问题三	—	7.09×10^{-16}

所有残差均远小于 1% 的判据，且图 15 中储量变化曲线与边界通量积分曲线重合，说明离散方程满足全局守恒，边界通量符号和表面控制体更新也相互一致。

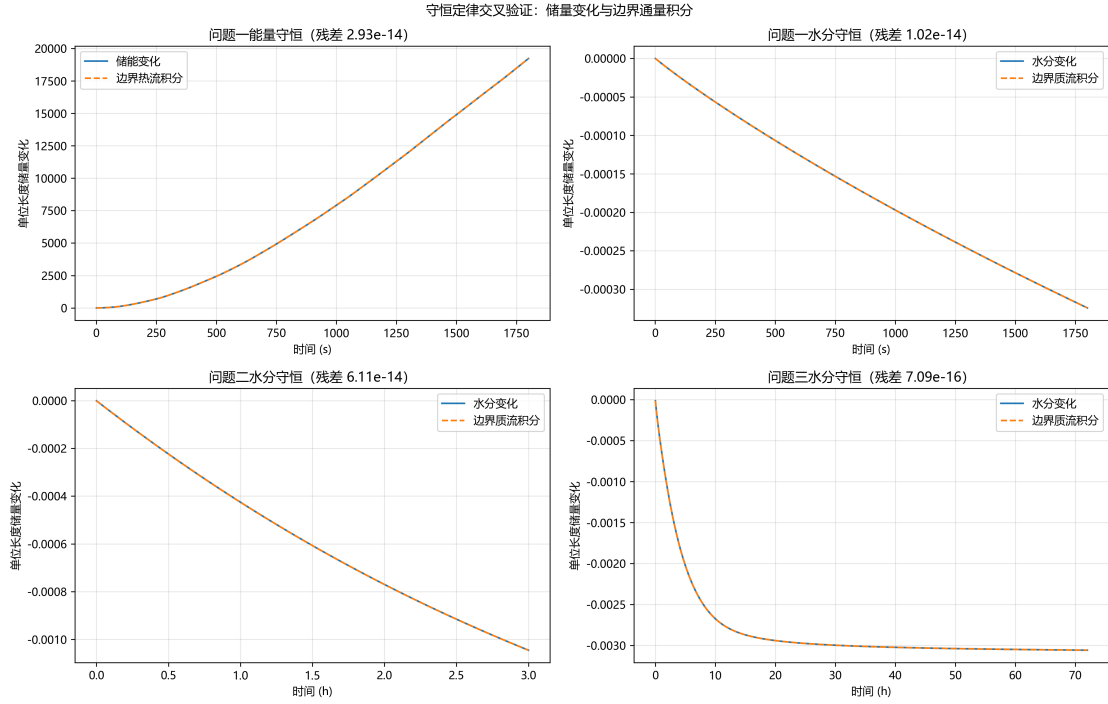


图 15: 储量变化与边界通量积分的守恒对照

8.1.4 物理范围与问题三阈值

程序在每个时间步检查数组有限性、水分浓度非负以及温度未触及截断范围。问题二全时段得到 $46.0518 \leq T \leq 50.0415$ °C, $0.7281 \leq C \leq 2.3745$; 问题三得到 $28.0 \leq T \leq 50.0392$ °C, $0.0517 \leq C \leq 2.55$, 未出现 NaN、无穷值或负水分。

问题三以全域最大水分浓度 $C_{\max}(t) = \max_i C_i(t)$ 作为结束判据，并检查其单调不增性。阈值前后相邻 60 s 网格点分别为 $C_b = 0.150009768$ 和 $C_a = 0.149992897$ 。采用线性插值

$$t_d = t_b + \frac{0.15 - C_b}{C_a - C_b}(t_a - t_b) \quad (66)$$

得到 $t_d = 210634.7398 \text{ s} = 58.50965 \text{ h}$ ，且插值点的 $C_{\max} = 0.150000000$ 。因此该时刻是满足全域水分浓度限制的最短数值时间。

8.1.5 问题四移动边界交叉验证

问题四的独立检查包括四项。第一，重新读取附件 2 并执行 PCHIP 插值，确认半径从 2.000 cm 单调降至 1.198 cm；第二，重新运行材料坐标求解器，与 result4.xlsx 的“无量纲网格”工作表逐元素比较，四位小数最大误差为 0；第三，检查恒温阶段全域最大水分浓度单调不增且无负值；第四，对离散方程在材料坐标上的积分恒等式进行逐步检查。令 w_i 为无量纲控制体权重，则水分方程给出

$$\sum_i w_i (C_i^{n+1} - C_i^n) = \frac{2h_m \Delta t}{R_{n+1}} (C_{\text{air}}^{n+1} - C_s^{n+1}). \quad (67)$$

该检验与程序推进方程使用同一时间层，最大绝对残差为 2.21×10^{-16} 。这里的量是材料坐标下的归一化储量；若将 C 解释为体积浓度，还需在守恒方程中另行加入收缩引起的稀释项，本文不作该解释。

采用 $\Delta t = 120, 60, 30$ s 的短时重算，误差比为 0.5641；采用 $\Delta \xi = 0.1, 0.05, 0.025$ 的短时重算，误差比为 0.2615。两者均随网格细化而下降，证明半径变化对应的 $R(t)^{-2}$ 和表面 $R(t)^{-1}$ 系数已被正确加入。第四问阈值前后的相邻网格点为 $C_{\text{max}}(184800 \text{ s}) = 0.15000799$ 和 $C_{\text{max}}(184860 \text{ s}) = 0.14997752$ 。线性插值得到 $184815.7394 \text{ s} = 51.33771 \text{ h}$ ，与结果表的结束行一致。

表 4: 问题四交叉验证结果

检验项目	数值结果
半径范围	2.000–1.198 cm，单调不增
Excel 与独立重算最大误差	0
材料坐标积分恒等式最大残差	2.21×10^{-16}
时间步收敛比（短时）	0.5641
空间步收敛比（短时）	0.2615
烘干结束时间	51.33771 h

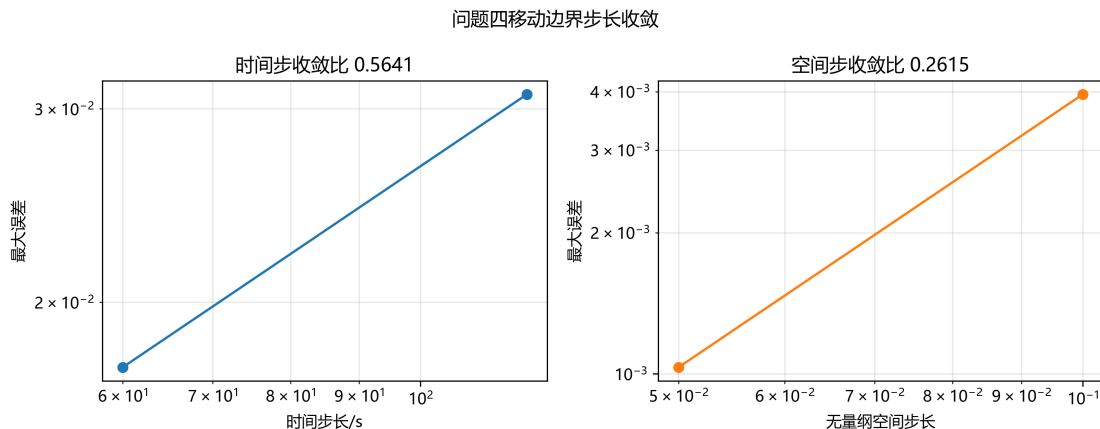


图 16: 问题四移动边界时间步与空间步收敛

8.2 误差分析

数值误差与模型误差分别讨论。由表 2 可知，时间步减半后误差约减半，空间步减半后误差约变为四分之一，故在当前网格下时间离散误差和空间离散误差均已进入稳定收敛区。结果写入 Excel 时统一四舍五入到四位小数，单个数值的舍入误差不超过 5×10^{-5} ；5500 s 断点比较的最大误差也小于 10^{-4} 。

模型误差主要来自连续介质假设、圆柱一维径向化、Robin 边界系数和经验物性公式的适用范围，以及环境数据插值。由于题目未提供完整实验时空场，本文不能用实验真值给出统计置信区间，因此采用独立重算、收敛性、守恒性和物理约束四类互相独立的证据评价结果可靠性。这些检验分别对应“文件是否写对”“离散是否收敛”“方程是否守恒”和“结果是否符合物理”，共同构成对三问数值答案的交叉证明。

9 模型的评价

9.1 模型的优点

- 优点 1
- 优点 2
- 优点 3

9.2 模型的推广与改进

9.3 模型的缺点

- 缺点 1
- 缺点 2

参考文献

A 文件列表

文件名	功能描述
q1_1.py	问题一边界插值
q1_2.py	问题一常物性求解与结果输出
q1_3.py	问题一收敛和守恒检验
q2_1.py-q2_3.py	问题二恒温边界、5500 s 初值和变物性求解
q3.py	问题三阈值判定与全过程输出
q4.py	问题四半径插值、材料坐标和附录 4 物性求解
validate_all.py	四问独立重算与交叉验证

B 代码

q1_1.py

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.interpolate import PchipInterpolator
5 from pathlib import Path
6
7 # 文件路径
8 BASE_DIR = Path(__file__).resolve().parent
9 input_file = BASE_DIR / 'input_excel' / '附件1.xlsx'
10 output_file = BASE_DIR / 'output_excel' / 'output1.xlsx'
11 output_file.parent.mkdir(parents=True, exist_ok=True)
12
13 # 读取边界数据
14 df = pd.read_excel(input_file, header=0)
15 t_orig = df.iloc[:, 0].values.astype(float)
16 temp_orig = df.iloc[:, 1].values.astype(float)
17 moist_orig = df.iloc[:, 2].values.astype(float)
18
19 # 生成1 s时间网格
20 t_new = np.arange(t_orig.min(), t_orig.max() + 0.1, 1.0)
21
22 # PCHIP插值
23 temp_new = PchipInterpolator(t_orig, temp_orig)(t_new)
24 moist_new = PchipInterpolator(t_orig, moist_orig)(t_new)
```

```

25
26 # 保存边界结果
27 pd.DataFrame({
28     '时间': t_new,
29     '温度': temp_new,
30     '水分浓度': moist_new
31 }).to_excel(output_file, index=False)
32
33 print(f"完成, 已保存至 {output_file}")
34
35 # 温度曲线
36 plt.figure()
37 plt.plot(t_new, temp_new, label='插值数据 (1s)')
38 plt.plot(t_orig, temp_orig, 'o', label='原始数据 (60s)')
39 plt.xlabel('时间 (s)')
40 plt.ylabel('温度')
41 plt.legend()
42 plt.show()
43
44 # 水分曲线
45 plt.figure()
46 plt.plot(t_new, moist_new, label='插值数据 (1s)')
47 plt.plot(t_orig, moist_orig, 'o', label='原始数据 (60s)')
48 plt.xlabel('时间 (s)')
49 plt.ylabel('水分浓度')
50 plt.legend()
51 plt.show()

```

q1_2.py

```

1 import pandas as pd
2 import numpy as np
3 from pathlib import Path
4
5 BASE_DIR = Path(__file__).resolve().parent
6 OUTPUT_DIR = BASE_DIR / 'output_excel'
7
8 # 读取边界数据
9 file_name = OUTPUT_DIR / 'output1.xlsx'
10 if not file_name.exists():
11     raise FileNotFoundError(f"未找到边界文件: {file_name}, 请先
        运行 q1_1.py")
12 df_input = pd.read_excel(file_name)

```

```

13 t_array = df_input.iloc[:, 0].values.astype(float)
14 T_air = df_input.iloc[:, 1].values.astype(float)
15 C_air = df_input.iloc[:, 2].values.astype(float)
16 if len(t_array) < 1801:
17     raise ValueError("边界文件至少需要覆盖 0~1800 s")
18
19 # 物性参数
20 rho, Cp, k = 820.0, 2600.0, 0.36
21 h, hm = 25.0, 8e-7
22 alpha = k / (rho * Cp)
23 Nt, Nr = 1801, 21
24 dt, dr = 1.0, 0.001
25 r_grid = np.linspace(0, 2, Nr)
26
27 T_matrix, C_matrix = np.zeros((Nt, Nr)), np.zeros((Nt, Nr))
28 T_matrix[0, :] = 28.0
29 C_matrix[0, :] = 2.55
30
31 # 水分扩散系数
32 def D_of_C(C_val):
33     C_val = np.clip(C_val, 1e-5, 10.0)
34     return 7e-9 * np.exp(-0.89 / C_val)
35
36 # 界面扩散系数
37 def D_face(C_left, C_right):
38     return D_of_C(0.5 * (C_left + C_right))
39
40 print("开始求解问题一...")
41
42 for n in range(0, Nt - 1):
43     T_air_next, C_air_next = T_air[n+1], C_air[n+1]
44
45     for i in range(1, Nr - 1):
46         r_val = i * dr
47         T_matrix[n+1, i] = T_matrix[n, i] + alpha * dt / (dr
48             **2) * (
49             (1 + dr/(2*r_val)) * T_matrix[n, i+1]
50             - 2 * T_matrix[n, i]
51             + (1 - dr/(2*r_val)) * T_matrix[n, i-1])
52
53     T_matrix[n+1, 0] = T_matrix[n, 0] + 4 * alpha * dt / (dr
54         **2) * (T_matrix[n, 1] - T_matrix[n, 0])

```

```

53
54 # 表面温度
55 i_s = Nr - 1
56 V_factor = i_s - 0.25
57 T_matrix[n+1, i_s] = T_matrix[n, i_s] + dt * (
58     2 * (i_s - 0.5) * k * (T_matrix[n, i_s - 1] - T_matrix[
59         n, i_s])
60     / (rho * Cp * V_factor * dr**2)
61     + 2 * i_s * h * (T_air_next - T_matrix[n, i_s])
62     / (rho * Cp * V_factor * dr)
63 )
64 # 水分内部节点
65 for i in range(1, Nr - 1):
66     r_val = i * dr
67     # 两侧界面系数
68     D_L = D_face(C_matrix[n, i-1], C_matrix[n, i])
69     D_R = D_face(C_matrix[n, i], C_matrix[n, i+1])
70     # 水分通量更新
71     C_matrix[n+1, i] = C_matrix[n, i] + dt / (dr**2) * (
72         (1 + dr/(2*r_val)) * D_R * (C_matrix[n, i+1] -
73             C_matrix[n, i])
74         + (1 - dr/(2*r_val)) * D_L * (C_matrix[n, i-1] -
75             C_matrix[n, i]))
76
77 # 水分中心节点
78 D_01 = D_face(C_matrix[n, 0], C_matrix[n, 1])
79 C_matrix[n+1, 0] = C_matrix[n, 0] + 4 * D_01 * dt / (dr**2)
80     * (C_matrix[n, 1] - C_matrix[n, 0])
81
82 # 水分表面节点
83 i_s = Nr - 1
84 D_sm = D_face(C_matrix[n, i_s-1], C_matrix[n, i_s]) # 表
85     面界面系数
86 V_factor = i_s - 0.25 # 表
87     面控制体系数
88 C_matrix[n+1, i_s] = C_matrix[n, i_s] + dt * (
89     2 * (i_s - 0.5) * D_sm * (C_matrix[n, i_s-1] - C_matrix[
90         n, i_s]) / (V_factor * dr**2)
91     + 2 * i_s * hm * (C_air_next - C_matrix[n, i_s]) / (
92         V_factor * dr)
93 )

```

```

87
88 print("问题一求解完成")
89
90 # 整理完整结果
91 col_names = [f"{x:.1f}" for x in r_grid]
92 time_col = np.arange(0, 1801, 1)
93
94 df_q1_1 = pd.DataFrame(np.round(T_matrix, 4), columns=col_names
95 )
96 df_q1_1.insert(0, '时间', time_col)
97
98 df_q1_2 = pd.DataFrame(np.round(C_matrix, 4), columns=col_names
99 )
100 df_q1_2.insert(0, '时间', time_col)
101
102 # 提取指定时刻
103 target_times = [100, 300, 600, 900, 1200, 1500, 1800]
104 target_radaii = ['0.0', '0.5', '1.0', '1.5', '2.0']
105
106 df_q1_3 = df_q1_1[df_q1_1['时间'].isin(target_times)][['时间']
107 + target_radaii].reset_index(drop=True)
108 df_q1_4 = df_q1_2[df_q1_2['时间'].isin(target_times)][['时间']
109 + target_radaii].reset_index(drop=True)
110
111 # 保存四个结果表
112 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
113 for name, frame in {
114     'q1_1.xlsx': df_q1_1,
115     'q1_2.xlsx': df_q1_2,
116     'q1_3.xlsx': df_q1_3,
117     'q1_4.xlsx': df_q1_4,
118 }.items():
119     try:
120         frame.to_excel(OUTPUT_DIR / name, index=False)
121     except PermissionError:
122         pending = OUTPUT_DIR / name.replace('.xlsx', '_pending.
123             xlsx')
124         frame.to_excel(pending, index=False)
125         print(f'文件被占用, 已写入临时结果: {pending}')
126
127 # 保存双工作表文件
128 result1_file = OUTPUT_DIR / 'result1.xlsx'

```

```

124 try:
125     with pd.ExcelWriter(result1_file) as writer:
126         df_q1_1.to_excel(writer, sheet_name='温度', index=False)
127         df_q1_2.to_excel(writer, sheet_name='水分浓度', index=False)
128         print(f'题目正式结果已保存: {result1_file}')
129 except PermissionError:
130     pending = OUTPUT_DIR / 'result1_pending.xlsx'
131     with pd.ExcelWriter(pending) as writer:
132         df_q1_1.to_excel(writer, sheet_name='温度', index=False)
133         df_q1_2.to_excel(writer, sheet_name='水分浓度', index=False)
134         print(f'文件被占用, 已写入临时结果: {pending}')
135
136 print("处理完成! 已覆盖并生成 q1_1.xlsx, q1_2.xlsx, q1_3.xlsx, q1_4.xlsx")
137 print("结果图请运行 plot_condensed.py 统一生成。")

```

q1_3.py

```

1 # 时间步、空间步和守恒检验
2 import pandas as pd
3 import numpy as np
4 from scipy.interpolate import PchipInterpolator
5 from pathlib import Path
6
7 BASE_DIR = Path(__file__).resolve().parent
8 OUTPUT_DIR = BASE_DIR / 'output_excel'
9
10
11 # 读取边界函数
12 def get_boundary_funcs():
13     path = OUTPUT_DIR / 'output1.xlsx'
14     df = pd.read_excel(path)
15     t_orig = df.iloc[:, 0].values.astype(float)
16     T_orig = df.iloc[:, 1].values.astype(float)
17     C_orig = df.iloc[:, 2].values.astype(float)
18     if np.any(np.diff(t_orig) <= 0):
19         raise ValueError('output1.xlsx 时间必须严格递增')
20     T_func = PchipInterpolator(t_orig, T_orig, extrapolate=False)

```

```

21     C_func = PchipInterpolator(t_orig, C_orig, extrapolate=
        False)
22     print(f'已读取 {path} 作为边界条件')
23     return T_func, C_func
24
25 # 求解温度 and 水分
26 def solve_pde(dt, dr, r_max=0.02, t_end=1800.0,
27              T_air_func=None, C_air_func=None):
28     if T_air_func is None or C_air_func is None:
29         T_air_func, C_air_func = get_boundary_funcs()
30
31     rho, Cp, k = 820.0, 2600.0, 0.36
32     h, hm = 25.0, 8e-7
33     alpha = k / (rho * Cp)
34
35     Nr = int(round(r_max / dr)) + 1
36     Nt = int(round(t_end / dt)) + 1
37
38     r = np.linspace(0, r_max, Nr)
39     t = np.linspace(0, t_end, Nt)
40
41     T = np.zeros((Nt, Nr))
42     C = np.zeros((Nt, Nr))
43     T[0, :] = 28.0
44     C[0, :] = 2.55
45
46     r_inner = r[1:-1]
47     coef_p = 1.0 + dr / (2.0 * r_inner)    # 外侧几何系数
48     coef_m = 1.0 - dr / (2.0 * r_inner)    # 内侧几何系数
49
50 # 水分扩散系数
51 def D_of_C(Cval):
52     return 7e-9 * np.exp(-0.89 / np.clip(Cval, 1e-5, 10.0))
53
54     stab_T = 4.0 * alpha * dt / dr**2
55     if stab_T >= 1.0:
56         print(f'    [!] dt={dt}, dr={dr} 时  $4*alpha*dt/dr^2 = \{$ 
            stab_T:.3f $\} >= 1$ , 可能不稳定')
57
58     for n in range(Nt - 1):
59         T_air_next = float(T_air_func(t[n + 1]))
60         C_air_next = float(C_air_func(t[n + 1]))

```

```

61
62     # 温度内部节点
63     T[n + 1, 1:-1] = T[n, 1:-1] + alpha * dt / dr**2 * (
64         coef_p * T[n, 2:] - 2.0 * T[n, 1:-1] + coef_m * T[n
65         , :-2]
66     )
67     # 水分内部节点
68     C_inner = C[n, 1:-1]
69     C_left = C[n, :-2]
70     C_right = C[n, 2:]
71     D_L = D_of_C(0.5 * (C_left + C_inner))    # 左界面系数
72     D_R = D_of_C(0.5 * (C_inner + C_right))    # 右界面系数
73     C[n + 1, 1:-1] = C_inner + dt / dr**2 * (
74         coef_p * D_R * (C_right - C_inner)
75         + coef_m * D_L * (C_left - C_inner)
76     )
77
78     # 温度中心节点
79     T[n + 1, 0] = T[n, 0] + 4.0 * alpha * dt / dr**2 * (T[n
80         , 1] - T[n, 0])
81
82     # 水分中心节点
83     D_01 = D_of_C(0.5 * (C[n, 0] + C[n, 1]))
84     C[n + 1, 0] = C[n, 0] + 4.0 * D_01 * dt / dr**2 * (C[n,
85         1] - C[n, 0])
86
87     # 温度表面节点
88     i_s = Nr - 1
89     V_factor = i_s - 0.25
90     T[n + 1, -1] = T[n, -1] + dt * (
91         2.0 * (i_s - 0.5) * k * (T[n, i_s - 1] - T[n, i_s])
92         / (rho * Cp * V_factor * dr**2)
93         + 2.0 * i_s * h * (T_air_next - T[n, i_s])
94         / (rho * Cp * V_factor * dr)
95     )
96
97     # 水分表面节点
98     i_s = Nr - 1
99     D_sm = D_of_C(0.5 * (C[n, i_s - 1] + C[n, i_s]))    # 表
100     面界面系数
101     V_factor = i_s - 0.25    #

```

表面控制体系数

```
99     C[n + 1, -1] = C[n, -1] + dt * (  
100         2.0 * (i_s - 0.5) * D_sm * (C[n, i_s - 1] - C[n,  
101             -1]) / (V_factor * dr**2)  
102         + 2.0 * i_s * hm * (C_air_next - C[n, -1]) / (  
103             V_factor * dr)  
104     )  
105  
106     return t, r, T, C  
107  
108 # 时间步收敛  
109 def verify_time_step():  
110     print('\n时间步收敛验证')  
111  
112     T_func, C_func = get_boundary_funcs()  
113     dr = 0.001  
114     dt_list = [1.0, 0.5, 0.25]  
115     results = {}  
116  
117     for dt in dt_list:  
118         print(f' 求解 dt = {dt} s ...')  
119         t, r, T, C = solve_pde(dt, dr, T_air_func=T_func,  
120             C_air_func=C_func)  
121         results[dt] = (t, r, T, C)  
122  
123 # 比较共同时间点  
124 common_t = np.arange(0, 1801, 100)  
125 T_common, C_common = {}, {}  
126 for dt in dt_list:  
127     t, r, T, C = results[dt]  
128     idx = np.round(common_t / dt).astype(int)  
129     T_common[dt] = T[idx, :]  
130     C_common[dt] = C[idx, :]  
131  
132     err_T_1_05 = np.max(np.abs(T_common[1.0] - T_common[0.5]))  
133     err_T_05_025 = np.max(np.abs(T_common[0.5] - T_common  
134         [0.25]))  
135  
136     err_C_1_05 = np.max(np.abs(C_common[1.0] - C_common[0.5]))  
137     err_C_05_025 = np.max(np.abs(C_common[0.5] - C_common  
138         [0.25]))  
139  
140     print('\n误差结果: ')
```

```

135     print(f'    温度 dt=1.0 vs 0.5    : {err_T_1_05:.6f} °C')
136     print(f'    温度 dt=0.5 vs 0.25   : {err_T_05_025:.6f} °C')
137     print(f'    水分 dt=1.0 vs 0.5    : {err_C_1_05:.6f}')
138     print(f'    水分 dt=0.5 vs 0.25   : {err_C_05_025:.6f}')
139     print('\n判断标准: ')
140     print('    1) 第二次误差应小于第一次')
141     print('    2) 收敛比应接近 0.5 (一阶)')
142     print(f'        温度收敛比 = {err_T_05_025/err_T_1_05:.3f}')
143     print(f'        水分收敛比 = {err_C_05_025/err_C_1_05:.3f}')
144     print('    3) 温度误差 < 0.1°C, 水分误差 < 0.01')
145     ratio_T = err_T_05_025 / err_T_1_05
146     ratio_C = err_C_05_025 / err_C_1_05
147     if not (err_T_05_025 < err_T_1_05 and err_C_05_025 <
148             err_C_1_05
149             and 0.35 < ratio_T < 0.65 and 0.35 < ratio_C < 0.65
150             and err_T_1_05 < 0.1 and err_C_1_05 < 0.01):
151         raise AssertionError('问题一时间步收敛未通过')
152 # 空间步收敛
153 def verify_space_step():
154     print('\n空间步长收敛验证')
155
156     T_func, C_func = get_boundary_funcs()
157     dt = 0.05
158     dr_list = [0.001, 0.0005, 0.00025]
159     results = {}
160
161     for dr in dr_list:
162         print(f'    求解 dr = {dr} m ...')
163         t, r, T, C = solve_pde(dt, dr, T_air_func=T_func,
164                                C_air_func=C_func)
165         results[dr] = (t, r, T, C)
166
167     # 比较共同半径点
168     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
169     T_final, C_final = {}, {}
170     for dr in dr_list:
171         t, r, T, C = results[dr]
172         T_final[dr] = np.interp(common_r, r, T[-1, :])
173         C_final[dr] = np.interp(common_r, r, C[-1, :])
174
175     err_T_1_05 = np.max(np.abs(T_final[0.001] - T_final

```

```

    [0.0005]))
175 err_T_05_025 = np.max(np.abs(T_final[0.0005] - T_final
    [0.00025]))
176 err_C_1_05 = np.max(np.abs(C_final[0.001] - C_final
    [0.0005]))
177 err_C_05_025 = np.max(np.abs(C_final[0.0005] - C_final
    [0.00025]))

178
179 print('\n误差结果: ')
180 print(f'  温度 dr=1mm vs 0.5mm      : {err_T_1_05:.6f} °C')
181 print(f'  温度 dr=0.5mm vs 0.25mm : {err_T_05_025:.6f} °C')
182 print(f'  水分 dr=1mm vs 0.5mm      : {err_C_1_05:.6f}')
183 print(f'  水分 dr=0.5mm vs 0.25mm : {err_C_05_025:.6f}')
184 print('\n判断标准: ')
185 print('  1) 第二次误差应小于第一次')
186 print('  2) 收敛比应接近 0.25 (二阶)')
187 print(f'      温度收敛比 = {err_T_05_025/err_T_1_05:.3f}')
188 print(f'      水分收敛比 = {err_C_05_025/err_C_1_05:.3f}')
189 print('  3) 温度差异 < 0.05°C, 水分差异 < 0.005')
190 ratio_T = err_T_05_025 / err_T_1_05
191 ratio_C = err_C_05_025 / err_C_1_05
192 if not (err_T_05_025 < err_T_1_05 and err_C_05_025 <
    err_C_1_05
193         and 0.15 < ratio_T < 0.35 and 0.15 < ratio_C < 0.35
194         and err_T_1_05 < 0.05 and err_C_1_05 < 0.005):
195     raise AssertionError('问题一空间步收敛未通过')
196
197 # 守恒检验
198 def verify_conservation():
199     print('\n守恒检查')
200
201     T_func, C_func = get_boundary_funcs()
202     dt = 1.0
203     dr = 0.001
204     r_max = 0.02
205     t_end = 1800.0
206
207     t, r, T, C = solve_pde(dt, dr, r_max, t_end, T_func, C_func
        )
208     Nt, Nr = T.shape
209
210     rho, Cp = 820.0, 2600.0

```

```

211     h, hm = 25.0, 8e-7
212
213     # 控制体体积
214     V = np.zeros(Nr)
215     V[0] = np.pi * (dr / 2.0)**2
216     for i in range(1, Nr - 1):
217         V[i] = 2.0 * np.pi * (i * dr) * dr
218     r_surf = (Nr - 1) * dr
219     V[-1] = np.pi * (r_surf**2 - (r_surf - dr / 2.0)**2)
220
221     # 总能量和总水分
222     Q = np.sum(rho * Cp * T * V, axis=1)
223     M = np.sum(C * V, axis=1)
224
225     A_surf = 2.0 * np.pi * r_surf
226     T_air_arr = np.array([float(T_func(tt)) for tt in t])
227     C_air_arr = np.array([float(C_func(tt)) for tt in t])
228
229     # 计算表面通量
230     q_step = h * A_surf * (T_air_arr[1:] - T[:-1, -1])
231     m_step = hm * A_surf * (C_air_arr[1:] - C[:-1, -1])
232     Q_flux_int = np.concatenate([[0.0], np.cumsum(q_step * dt)
233                                   ])
234
235     M_flux_int = np.concatenate([[0.0], np.cumsum(m_step * dt)
236                                   ])
237
238     Q_change = Q - Q[0]
239     M_change = M - M[0]
240
241     eps = 1e-12
242     rel_err_Q = np.abs(Q_change - Q_flux_int) / np.maximum(np.
243                     abs(Q_change), eps)
244     rel_err_M = np.abs(M_change - M_flux_int) / np.maximum(np.
245                     abs(M_change), eps)
246
247     print('\n误差结果: ')
248     print(f' 最终能量守恒相对误差 : {rel_err_Q[-1]:.4%}')
249     print(f' 最终水分守恒相对误差 : {rel_err_M[-1]:.4%}')
250     print('\n判断标准: ')
251     print(' 1) 两条累积曲线应基本重合')
252     print(' 2) 相对误差应 < 1%~5%')
253     print(' 3) 若误差快速累积, 检查边界离散或体积权重')

```

```

249     if rel_err_Q[-1] >= 0.01 or rel_err_M[-1] >= 0.01:
250         raise AssertionError('问题一守恒检验未通过')
251
252 if __name__ == '__main__':
253     verify_time_step()
254     verify_space_step()
255     verify_conservation()
256     print('\n全部验证完成。结果图请运行 plot_condensed.py, 验证
        图请运行 validate_all.py。')

```

q2_3.py

```

1  # 问题二恒温段求解和检验
2
3  from pathlib import Path
4
5  import numpy as np
6  import pandas as pd
7
8  BASE_DIR = Path(__file__).resolve().parent
9  OUTPUT_DIR = BASE_DIR / "output_excel"
10 DEFAULT_INITIAL_FILE = OUTPUT_DIR / "output3.xlsx"
11 DEFAULT_BOUNDARY_FILE = OUTPUT_DIR / "output2.xlsx"
12 INITIAL_TIME = 5500.0
13 DEFAULT_SIMULATION_TIME = 10800.0  # 5500 s 后计算3小时
14
15
16 def _as_scalar_or_array(value):
17     arr = np.asarray(value, dtype=float)
18     return float(arr) if arr.ndim == 0 else arr
19
20
21 def get_boundary_funcs(path=DEFAULT_BOUNDARY_FILE, t_start=
    INITIAL_TIME):
22     # 读取边界
23     path = Path(path)
24     if not path.is_absolute():
25         path = BASE_DIR / path
26     if not path.exists():
27         raise FileNotFoundError(f"Boundary file not found: {
            path}")
28
29     df = pd.read_excel(path)

```

```

30     if df.shape[1] < 3 or len(df) < 2:
31         raise ValueError("Boundary file must contain at least 2
                               rows and 3 columns")
32     t_data = df.iloc[:, 0].to_numpy(dtype=float)
33     T_data = df.iloc[:, 1].to_numpy(dtype=float)
34     C_data = df.iloc[:, 2].to_numpy(dtype=float)
35     if not (np.all(np.isfinite(t_data)) and np.all(np.isfinite(
        T_data))
36             and np.all(np.isfinite(C_data))):
37         raise ValueError("Boundary data contains NaN or
                               infinite values")
38     if np.any(np.diff(t_data) <= 0):
39         raise ValueError("Boundary time must be strictly
                               increasing")
40     if t_start < t_data[0] - 1e-9:
41         raise ValueError(
42             f"Simulation starts at {t_start:g} s, but boundary
               data starts at "
43             f"{t_data[0]:g} s. Do not extrapolate backward."
44         )
45
46     tail_n = min(1000, len(t_data))
47     T_tail = float(np.mean(T_data[-tail_n:]))
48     C_tail = float(np.mean(C_data[-tail_n:]))
49
50     def make_func(values, tail_value):
51         def func(query):
52             q = np.asarray(query, dtype=float)
53             if np.any(q < t_data[0] - 1e-9):
54                 raise ValueError(
55                     f"Boundary queried before available data:
                       min={np.min(q):g} s, "
56                     f"first={t_data[0]:g} s"
57                 )
58             result = np.interp(q, t_data, values)
59             result = np.where(q > t_data[-1], tail_value,
                               result)
60             return _as_scalar_or_array(result)
61
62         return func
63
64     T_func = make_func(T_data, T_tail)

```

```

65     C_func = make_func(C_data, C_tail)
66     print(
67         f"Loaded boundary: {path.name}, range={t_data[0]:g}..{
            t_data[-1]:g} s, "
68         f"tail=({T_tail:.4f} C, {C_tail:.6f})"
69     )
70     return T_func, C_func
71
72
73 def load_initial_profile(path=DEFAULT_INITIAL_FILE, r_max=0.02,
74     Nr=21):
75     # 读取5500 s剖面
76     path = Path(path)
77     if not path.is_absolute():
78         path = BASE_DIR / path
79
80     if not path.exists():
81         raise FileNotFoundError(f"Initial profile file not
82             found: {path}")
83
84     df = pd.read_excel(path)
85
86     if df.shape[1] < 3:
87         raise ValueError("output3.xlsx必须包含三列: r,T,C")
88
89     radius_cm = df.iloc[:, 0].to_numpy(dtype=float)
90     T0_data = df.iloc[:, 1].to_numpy(dtype=float)
91     C0_data = df.iloc[:, 2].to_numpy(dtype=float)
92
93     # 删除空值
94     mask = np.isfinite(radius_cm) & np.isfinite(T0_data) & np.
95         isfinite(C0_data)
96     radius_cm = radius_cm[mask]
97     T0_data = T0_data[mask]
98     C0_data = C0_data[mask]
99
100     if len(radius_cm) < 2 or np.any(np.diff(radius_cm) <= 0):
101         raise ValueError("output3.xlsx半径必须严格递增")
102
103     # 检查半径
104     if radius_cm[0] > 1e-12:
105         raise ValueError("output3.xlsx第一行半径必须为0cm")

```

```

103
104 # 补齐2.0 cm节点
105 if radius_cm[-1] < r_max * 100 - 1e-12:
106     if abs(radius_cm[-1] - (r_max*100-0.1)) < 1e-8:
107         radius_cm = np.append(radius_cm, r_max*100)
108         T0_data = np.append(T0_data, T0_data[-1])
109         C0_data = np.append(C0_data, C0_data[-1])
110     else:
111         raise ValueError(
112             f"output3.xlsx半径范围不足: {radius_cm[0]}~{
113                 radius_cm[-1]} cm"
114         )
115
116 radius_m = radius_cm / 100.0
117
118 r = np.linspace(0.0, r_max, Nr)
119
120 T0 = np.interp(r, radius_m, T0_data)
121 C0 = np.interp(r, radius_m, C0_data)
122
123 print(
124     f"Loaded initial profile: {path.name}"
125 )
126 print(
127     f"radius: {radius_cm[0]:.1f}~{radius_cm[-1]:.1f} cm, "
128     f"nodes={len(radius_cm)}"
129 )
130 print(
131     f"T range={T0.min():.4f}~{T0.max():.4f} C, "
132     f"C range={C0.min():.6f}~{C0.max():.6f} kg/kg"
133 )
134
135 return r, T0, C0
136
137 # 附录3物性公式
138 def rho_of_C(C_val):
139     return 650.0 + 128.0 * C_val
140
141
142 def cp_of_C(C_val):
143     return 1450.0 + 2736.0 * (C_val / (C_val + 1.0))

```

```

144
145
146 def k_of_C(C_val):
147     return 0.21 + 0.38 * (C_val / (C_val + 1.0))
148
149
150 def D_of_CT(C_val, T_val):
151     C_safe = np.clip(C_val, 0.05, 10.0)
152     T_K = np.clip(T_val + 273.15, 273.15, 373.15)
153     D = 2.4e-3 * np.exp(-0.45 / C_safe) * np.exp(-3850.0 / T_K)
154     return np.clip(D, 1e-14, 1e-3)
155
156
157 def solve_tridiag(a, b, c, d):
158     # 追赶法
159     n = len(b)
160     cp = np.zeros(n)
161     dp = np.zeros(n)
162     if abs(b[0]) < 1e-14:
163         raise ZeroDivisionError("Singular tridiagonal system at
164                                 first row")
165     cp[0] = c[0] / b[0]
166     dp[0] = d[0] / b[0]
167     for i in range(1, n):
168         m = b[i] - a[i] * cp[i - 1]
169         if abs(m) < 1e-14:
170             raise ZeroDivisionError(f"Singular tridiagonal
171                                     system at row {i}")
172         if i < n - 1:
173             cp[i] = c[i] / m
174             dp[i] = (d[i] - a[i] * dp[i - 1]) / m
175     x = np.zeros(n)
176     x[-1] = dp[-1]
177     for i in range(n - 2, -1, -1):
178         x[i] = dp[i] - cp[i] * x[i + 1]
179     return x
180
181 def solve_pde(
182     dt,
183     dr,
184     r_max=0.02,

```

```

184     t_end=DEFAULT_SIMULATION_TIME,
185     t_start=INITIAL_TIME,
186     initial_file=DEFAULT_INITIAL_FILE,
187     initial_T=None,
188     initial_C=None,
189     T_air_func=None,
190     C_air_func=None,
191 ):
192     # 求解恒温段
193     if dt <= 0 or dr <= 0 or r_max <= 0 or t_end < 0:
194         raise ValueError("dt, dr and r_max must be positive;
195                             t_end cannot be negative")
196     if (T_air_func is None) != (C_air_func is None):
197         raise ValueError("Provide both T_air_func and
198                             C_air_func, or neither")
199     if T_air_func is None:
200         T_air_func, C_air_func = get_boundary_funcs(t_start=
201             t_start)
202
203     Nr = int(round(r_max / dr)) + 1
204     Nt = int(round(t_end / dt)) + 1
205     r = np.linspace(0.0, r_max, Nr)
206     t = np.linspace(0.0, t_end, Nt)
207     if initial_T is None and initial_C is None:
208         r0, T0, C0 = load_initial_profile(initial_file, r_max=
209             r_max, Nr=Nr)
210         if not np.allclose(r0, r):
211             raise ValueError("Initial profile grid does not
212                             match solver grid")
213     elif initial_T is None or initial_C is None:
214         raise ValueError("initial_T 和 initial_C 必须同时提供")
215     else:
216         T0 = np.asarray(initial_T, dtype=float)
217         C0 = np.asarray(initial_C, dtype=float)
218         if T0.ndim == 0:
219             T0 = np.full(Nr, float(T0))
220         if C0.ndim == 0:
221             C0 = np.full(Nr, float(C0))
222         if T0.shape != (Nr,) or C0.shape != (Nr,):
223             raise ValueError(f"初始数组长度必须为 {Nr}")
224         if not (np.all(np.isfinite(T0)) and np.all(np.isfinite(
225             C0))):

```

```

220         raise ValueError("初始温度或水分含有 NaN/无穷值")
221
222     T = np.zeros((Nt, Nr))
223     C = np.zeros((Nt, Nr))
224     T[0, :] = T0
225     C[0, :] = C0
226
227     h, hm = 25.0, 8e-7
228     i_s = Nr - 1
229     V_factor = i_s - 0.25
230     r_inner = r[1:-1]
231     coef_p = 1.0 + dr / (2.0 * r_inner)
232     coef_m = 1.0 - dr / (2.0 * r_inner)
233
234     for n in range(Nt - 1):
235         T_air_next = float(T_air_func(t_start + t[n + 1]))
236         C_air_next = float(C_air_func(t_start + t[n + 1]))
237
238         C_mid_L = 0.5 * (C[n, :-2] + C[n, 1:-1])
239         C_mid_R = 0.5 * (C[n, 1:-1] + C[n, 2:])
240         T_mid_L = 0.5 * (T[n, :-2] + T[n, 1:-1])
241         T_mid_R = 0.5 * (T[n, 1:-1] + T[n, 2:])
242         k_L = k_of_C(C_mid_L)
243         k_R = k_of_C(C_mid_R)
244         D_L = D_of_CT(C_mid_L, T_mid_L)
245         D_R = D_of_CT(C_mid_R, T_mid_R)
246         rho_inner = rho_of_C(C[n, 1:-1])
247         cp_inner = cp_of_C(C[n, 1:-1])
248         alpha_T_inner = dt / (rho_inner * cp_inner * dr**2)
249
250         a_T = np.zeros(Nr)
251         b_T = np.zeros(Nr)
252         c_T = np.zeros(Nr)
253         d_T = np.zeros(Nr)
254         for idx in range(Nr - 2):
255             i = idx + 1
256             ai = alpha_T_inner[idx]
257             a_T[i] = -ai * coef_m[idx] * k_L[idx]
258             b_T[i] = 1.0 + ai * (coef_p[idx] * k_R[idx] +
259                                 coef_m[idx] * k_L[idx])
259             c_T[i] = -ai * coef_p[idx] * k_R[idx]
260             d_T[i] = T[n, i]

```

```

261
262     k_01 = k_of_C(0.5 * (C[n, 0] + C[n, 1]))
263     rho_0 = rho_of_C(C[n, 0])
264     cp_0 = cp_of_C(C[n, 0])
265     alpha_0 = dt / (rho_0 * cp_0 * dr**2)
266     b_T[0] = 1.0 + 4.0 * alpha_0 * k_01
267     c_T[0] = -4.0 * alpha_0 * k_01
268     d_T[0] = T[n, 0]
269
270     C_s_T = C[n, i_s]
271     rho_s_T = rho_of_C(C_s_T)
272     cp_s_T = cp_of_C(C_s_T)
273     # 表面平均物性
274     k_s_T = k_of_C(0.5 * (C[n, i_s - 1] + C[n, i_s]))
275     beta_T = dt / (rho_s_T * cp_s_T * V_factor * dr**2)
276     coef_s_T_diff = 2.0 * (i_s - 0.5) * k_s_T
277     coef_s_T_conv = 2.0 * i_s * h * dr
278     a_T[i_s] = -beta_T * coef_s_T_diff
279     b_T[i_s] = 1.0 + beta_T * (coef_s_T_diff +
280                               coef_s_T_conv)
281     d_T[i_s] = T[n, i_s] + beta_T * coef_s_T_conv *
282         T_air_next
283     T[n + 1, :] = solve_tridiag(a_T, b_T, c_T, d_T)
284
285     alpha_C_inner = dt / dr**2
286     a_C = np.zeros(Nr)
287     b_C = np.zeros(Nr)
288     c_C = np.zeros(Nr)
289     d_C = np.zeros(Nr)
290     for idx in range(Nr - 2):
291         i = idx + 1
292         a_C[i] = -alpha_C_inner * coef_m[idx] * D_L[idx]
293         b_C[i] = 1.0 + alpha_C_inner * (coef_p[idx] * D_R[
294             idx] + coef_m[idx] * D_L[idx])
295         c_C[i] = -alpha_C_inner * coef_p[idx] * D_R[idx]
296         d_C[i] = C[n, i]
297
298     D_01 = D_of_CT(0.5 * (C[n, 0] + C[n, 1]), 0.5 * (T[n,
299         0] + T[n, 1]))
300     b_C[0] = 1.0 + 4.0 * alpha_C_inner * D_01
301     c_C[0] = -4.0 * alpha_C_inner * D_01
302     d_C[0] = C[n, 0]

```

```

299
300     # 表面界面扩散系数
301     D_s_C = D_of_CT(
302         0.5 * (C[n, i_s - 1] + C[n, i_s]),
303         0.5 * (T[n, i_s - 1] + T[n, i_s]),
304     )
305     g1 = dt * 2.0 * (i_s - 0.5) * D_s_C / (V_factor * dr
306         **2)
307     g2 = dt * 2.0 * i_s * hm / (V_factor * dr)
308     a_C[i_s] = -g1
309     b_C[i_s] = 1.0 + g1 + g2
310     d_C[i_s] = C[n, i_s] + g2 * C_air_next
311     C[n + 1, :] = solve_tridiag(a_C, b_C, c_C, d_C)
312
313     C[n + 1, :] = np.clip(C[n + 1, :], 0.0, 10.0)
314     T[n + 1, :] = np.clip(T[n + 1, :], -50.0, 250.0)
315
316     return t, r, T, C
317
318 def _solve_for_validation(dt, dr, t_end=1800.0):
319     T_func, C_func = get_boundary_funcs(t_start=INITIAL_TIME)
320     return solve_pde(
321         dt, dr, t_end=t_end, t_start=INITIAL_TIME,
322         initial_file=DEFAULT_INITIAL_FILE,
323         T_air_func=T_func, C_air_func=C_func,
324     )
325
326
327 def verify_initial_profile():
328     # 检查初值
329     _, T0, C0 = load_initial_profile(DEFAULT_INITIAL_FILE,
330         r_max=0.02, Nr=21)
331     t, r, T, C = _solve_for_validation(1.0, 0.001, t_end=0.0)
332     err_T = float(np.max(np.abs(T[0] - T0)))
333     err_C = float(np.max(np.abs(C[0] - C0)))
334     print("\n初始剖面检查")
335     print(f"    max |T[0]-T_output3| = {err_T:.3e} C")
336     print(f"    max |C[0]-C_output3| = {err_C:.3e} kg/kg")
337     print(f"    initial radius grid = {r[0] * 100:g}...{r[-1] *
338         100:g} cm")
339     if err_T > 1e-10 or err_C > 1e-10:

```

```

338         raise AssertionError("Initial row does not match
                                output3.xlsx")
339     return t, r, T, C
340
341
342 def verify_time_step(t_end=1800.0):
343     print("\n时间步收敛")
344     dt_list = [1.0, 0.5, 0.25]
345     results = {dt: _solve_for_validation(dt, 0.001, t_end=t_end
        ) for dt in dt_list}
346     common_t = np.arange(0.0, t_end + 1e-9, 100.0)
347     T_common, C_common = {}, {}
348     for dt, result in results.items():
349         _, _, T, C = result
350         idx = np rint(common_t / dt).astype(int)
351         T_common[dt] = T[idx, :]
352         C_common[dt] = C[idx, :]
353     err_T_1_05 = np.max(np.abs(T_common[1.0] - T_common[0.5]))
354     err_T_05_025 = np.max(np.abs(T_common[0.5] - T_common
        [0.25]))
355     err_C_1_05 = np.max(np.abs(C_common[1.0] - C_common[0.5]))
356     err_C_05_025 = np.max(np.abs(C_common[0.5] - C_common
        [0.25]))
357     print(f" T: dt 1.0 vs 0.5 = {err_T_1_05:.6g} C")
358     print(f" T: dt 0.5 vs 0.25 = {err_T_05_025:.6g} C")
359     print(f" C: dt 1.0 vs 0.5 = {err_C_1_05:.6g}")
360     print(f" C: dt 0.5 vs 0.25 = {err_C_05_025:.6g}")
361     print(f" 温度收敛比 = {err_T_05_025 / err_T_1_05:.4f}")
362     print(f" 水分收敛比 = {err_C_05_025 / err_C_1_05:.4f}")
363     print(" 标准: 第二次误差更小, 收敛比接近 0.5")
364     ratio_T = err_T_05_025 / err_T_1_05
365     ratio_C = err_C_05_025 / err_C_1_05
366     if not (err_T_05_025 < err_T_1_05 and err_C_05_025 <
        err_C_1_05
367             and 0.35 < ratio_T < 0.65 and 0.35 < ratio_C < 0.65
368             and err_T_1_05 < 0.01 and err_C_1_05 < 0.01):
369         raise AssertionError("问题二时间步收敛未通过")
370     return results
371
372
373 def verify_space_step(t_end=1800.0):
374     print("\n空间步收敛")

```

```

375 dt = 1.0
376 dr_list = [0.001, 0.0005, 0.00025]
377 results = {dr: _solve_for_validation(dt, dr, t_end=t_end)
              for dr in dr_list}
378 common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
379 T_final, C_final = {}, {}
380 for dr, result in results.items():
381     _, r, T, C = result
382     T_final[dr] = np.interp(common_r, r, T[-1, :])
383     C_final[dr] = np.interp(common_r, r, C[-1, :])
384 err_T_1_05 = np.max(np.abs(T_final[0.001] - T_final
                             [0.0005]))
385 err_T_05_025 = np.max(np.abs(T_final[0.0005] - T_final
                             [0.00025]))
386 err_C_1_05 = np.max(np.abs(C_final[0.001] - C_final
                             [0.0005]))
387 err_C_05_025 = np.max(np.abs(C_final[0.0005] - C_final
                             [0.00025]))
388 print(f" T: dr 1.0 vs 0.5 mm = {err_T_1_05:.6g} C")
389 print(f" T: dr 0.5 vs 0.25 mm = {err_T_05_025:.6g} C")
390 print(f" C: dr 1.0 vs 0.5 mm = {err_C_1_05:.6g}")
391 print(f" C: dr 0.5 vs 0.25 mm = {err_C_05_025:.6g}")
392 print(f" 温度收敛比 = {err_T_05_025 / err_T_1_05:.4f}")
393 print(f" 水分收敛比 = {err_C_05_025 / err_C_1_05:.4f}")
394 print(" 标准: 第二次误差更小, 收敛比接近 0.25")
395 ratio_T = err_T_05_025 / err_T_1_05
396 ratio_C = err_C_05_025 / err_C_1_05
397 if not (err_T_05_025 < err_T_1_05 and err_C_05_025 <
          err_C_1_05
          and 0.15 < ratio_T < 0.35 and 0.15 < ratio_C < 0.35
          and err_T_1_05 < 0.01 and err_C_1_05 < 0.01):
400     raise AssertionError("问题二空间步收敛未通过")
401 return results
402
403
404 def _control_volumes(dr, Nr):
405     V = np.zeros(Nr)
406     V[0] = np.pi * (dr / 2.0) ** 2
407     for i in range(1, Nr - 1):
408         V[i] = 2.0 * np.pi * (i * dr) * dr
409     r_surf = (Nr - 1) * dr
410     V[-1] = np.pi * (r_surf**2 - (r_surf - dr / 2.0) ** 2)

```

```

411     return V, r_surf
412
413
414 def verify_conservation(t_end=DEFAULT_SIMULATION_TIME):
415     print("\n守恒检查")
416     dt, dr = 1.0, 0.001
417     T_func, C_func = get_boundary_funcs(t_start=INITIAL_TIME)
418     t, r, T, C = solve_pde(
419         dt, dr, t_end=t_end, t_start=INITIAL_TIME,
420         initial_file=DEFAULT_INITIAL_FILE,
421         T_air_func=T_func, C_air_func=C_func,
422     )
423     _, Nr = T.shape
424     V, r_surf = _control_volumes(dr, Nr)
425     h, hm = 25.0, 8e-7
426     A_surf = 2.0 * np.pi * r_surf
427
428     M = np.sum(C * V, axis=1)
429     # 表面离散通量
430     C_air_next = np.asarray(C_func(INITIAL_TIME + t[1:]), dtype
431                             =float)
432     m_flux_step = hm * A_surf * (C_air_next - C[1:, -1])
433     M_flux_int = np.concatenate([[0.0], np.cumsum(m_flux_step *
434     dt)])
435     M_change = M - M[0]
436     moisture_rel_err = abs(M_change[-1] - M_flux_int[-1]) / max
437     (abs(M_change[-1]), 1e-12)
438
439     # 计算蓄热量
440     thermal_storage = np.sum(
441         rho_of_C(C[:-1]) * cp_of_C(C[:-1]) * (T[1:] - T[:-1]) *
442         V,
443         axis=1,
444     )
445     T_air_next = np.asarray(T_func(INITIAL_TIME + t[1:]), dtype
446                             =float)
447     q_flux_step = h * A_surf * (T_air_next - T[1:, -1])
448     energy_residual = abs(np.sum(thermal_storage) - np.sum(
449         q_flux_step * dt))
450     energy_scale = max(abs(np.sum(thermal_storage)), 1e-12)
451     energy_rel_err = energy_residual / energy_scale
452

```

```

447     print(f" 最终水分守恒残差 = {moisture_rel_err:.4%}")
448     print(f" 最终热量诊断残差 = {energy_rel_err:.4%}")
449     print(" 标准: 水分和热量残差均小于 1%")
450     if moisture_rel_err >= 0.01 or energy_rel_err >= 0.01:
451         raise AssertionError("问题二守恒检验未通过")
452     return moisture_rel_err, energy_rel_err
453
454
455 def verify_physical_bounds(t_end=DEFAULT_SIMULATION_TIME):
456     print("\n物理范围检查")
457     t, r, T, C = _solve_for_validation(1.0, 0.001, t_end=t_end)
458     finite = bool(np.all(np.isfinite(T)) and np.all(np.isfinite
459                 (C)))
459     clipped_T = bool(np.any((T <= -50.0) | (T >= 250.0)))
460     clipped_C = bool(np.any((C <= 0.0) | (C >= 10.0)))
461     print(f" 数组有限: {finite}")
462     print(f" T 范围: {T.min():.6f}..{T.max():.6f} C; 触及截
463           断: {clipped_T}")
463     print(f" C 范围: {C.min():.6f}..{C.max():.6f} kg/kg; 触及
464           截断: {clipped_C}")
464     print(" 标准: 有限值、C>=0, 且计算未触及截断")
465     if not finite:
466         raise AssertionError("NaN or infinite value detected")
467     if np.any(C < -1e-12) or clipped_T or clipped_C:
468         raise AssertionError("问题二物理范围检验未通过")
469     return t, r, T, C
470
471
472 def save_problem2_results(
473     dt=1.0,
474     dr=0.001,
475     t_end=DEFAULT_SIMULATION_TIME,
476 ):
477     # 保存问题二结果
478     T_func, C_func = get_boundary_funcs(t_start=INITIAL_TIME)
479     t, r, T, C = solve_pde(
480         dt, dr, t_end=t_end, t_start=INITIAL_TIME,
481         initial_file=DEFAULT_INITIAL_FILE,
482         T_air_func=T_func, C_air_func=C_func,
483     )
484     radii_cm = [f"{value * 100.0:.1f}" for value in r]
485     time_s = np rint(t).astype(int)

```

```

486 df_T = pd.DataFrame(np.round(T, 4), columns=radii_cm)
487 df_C = pd.DataFrame(np.round(C, 4), columns=radii_cm)
488 df_T.insert(0, "时间", time_s)
489 df_C.insert(0, "时间", time_s)
490
491 target_hours = np.arange(0.5, 3.0 + 1e-12, 0.5)
492 target_seconds = np rint(target_hours * 3600.0).astype(int)
493 target_radii = ["0.0", "0.5", "1.0", "1.5", "2.0"]
494 # 取目标时刻
495 sample_idx = [int(np.argmin(np.abs(time_s - value))) for
    value in target_seconds]
496 df_T_sample = df_T.iloc[sample_idx][["时间"] + target_radii
    ].copy()
497 df_C_sample = df_C.iloc[sample_idx][["时间"] + target_radii
    ].copy()
498 # 摘要表用小时
499 df_T_sample["时间"] = df_T_sample["时间"] / 3600.0
500 df_C_sample["时间"] = df_C_sample["时间"] / 3600.0
501
502 if not np.allclose(df_T_sample["时间"].to_numpy(),
    target_hours):
503     raise AssertionError("温度摘要表时间点错误")
504 if not np.allclose(df_C_sample["时间"].to_numpy(),
    target_hours):
505     raise AssertionError("水分摘要表时间点错误")
506
507 outputs = {
508     "q2_1.xlsx": df_T,
509     "q2_2.xlsx": df_C,
510     "q2_3.xlsx": df_T_sample,
511     "q2_4.xlsx": df_C_sample,
512 }
513 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
514 for name, frame in outputs.items():
515     target = OUTPUT_DIR / name
516     try:
517         frame.to_excel(target, index=False)
518         print(f"已保存: {target}")
519     except PermissionError:
520         pending = OUTPUT_DIR / name.replace(".xlsx", "
        _pending.xlsx")
521         frame.to_excel(pending, index=False)

```

```

522         print(f"文件被占用，已写入临时结果：{pending}")
523
524     # 保存双工作表
525     result2_file = OUTPUT_DIR / "result2.xlsx"
526     try:
527         with pd.ExcelWriter(result2_file) as writer:
528             df_T.to_excel(writer, sheet_name="温度", index=
                    False)
529             df_C.to_excel(writer, sheet_name="水分浓度", index=
                    False)
530         print(f"题目正式结果已保存：{result2_file}")
531     except PermissionError:
532         pending = OUTPUT_DIR / "result2_pending.xlsx"
533         with pd.ExcelWriter(pending) as writer:
534             df_T.to_excel(writer, sheet_name="温度", index=
                    False)
535             df_C.to_excel(writer, sheet_name="水分浓度", index=
                    False)
536         print(f"文件被占用，已写入临时结果：{pending}")
537
538     print(f"完整表时间：0~{int(round(t_end))} s，共 {len(t)}
        行")
539     print("摘要表时间：0.5、1.0、1.5、2.0、2.5、3.0 h")
540     print(f"径向范围：0~{r[-1] * 100:g} cm，共 {len(r)} 个节点
        ")
541     return outputs, t, r, T, C
542
543
544 def plot_problem2_results(t, r, T, C):
545     # 结果图统一由 plot_condensed.py 生成
546     return None
547
548
549 if __name__ == "__main__":
550     _, t_result, r_result, T_result, C_result =
        save_problem2_results()
551     verify_initial_profile()
552     verify_time_step()
553     verify_space_step()
554     verify_conservation()
555     verify_physical_bounds()
556     print("\n问题二全部数值检验完成。")

```

q3.py

```
1 # 问题三干燥时间计算
2
3 from pathlib import Path
4
5 import numpy as np
6 import pandas as pd
7
8 from q2_3 import (
9     BASE_DIR,
10     DEFAULT_INITIAL_FILE,
11     OUTPUT_DIR,
12     INITIAL_TIME,
13     _control_volumes,
14     get_boundary_funcs,
15     load_initial_profile,
16     solve_pde,
17 )
18
19
20 MOISTURE_LIMIT = 0.15
21 MAX_ABSOLUTE_TIME = 259200.0 # 最长3天
22 DT = 60.0
23 DR = 0.001
24 R_MAX = 0.02
25
26
27 def get_full_boundary_funcs():
28     # 拼接两段边界
29     T_pre, C_pre = get_boundary_funcs(
30         path=OUTPUT_DIR / "output1.xlsx", t_start=0.0
31     )
32     stable_file = OUTPUT_DIR / "output2.xlsx"
33     if not stable_file.exists():
34         raise FileNotFoundError(f"未找到恒温边界: {stable_file}
35                                , 请先运行 q2_1.py")
36     T_stable, C_stable = get_boundary_funcs(
37         path=stable_file, t_start=INITIAL_TIME
38     )
39
40     def join(pre_func, stable_func):
41         def func(query):
```

```

41         q = np.asarray(query, dtype=float)
42         scalar = q.ndim == 0
43         q1 = np.atleast_1d(q)
44         if np.any(q1 < -1e-9):
45             raise ValueError("边界查询时间不能为负")
46         out = np.empty_like(q1)
47         pre_mask = q1 <= INITIAL_TIME
48         if np.any(pre_mask):
49             out[pre_mask] = np.asarray(pre_func(q1[pre_mask
50                                     ]), dtype=float)
51         if np.any(~pre_mask):
52             out[~pre_mask] = np.asarray(stable_func(q1[~
53                                     pre_mask])), dtype=float)
54         return float(out[0]) if scalar else out
55
56     return func
57
58     return join(T_pre, T_stable), join(C_pre, C_stable)
59
60 def get_simulation_end(max_absolute_time=MAX_ABSOLUTE_TIME, dt=
61     DT):
62     # 计算整步终点
63     if max_absolute_time < 0 or dt <= 0:
64         raise ValueError("时间参数必须为正")
65     return float(np.floor(max_absolute_time / dt) * dt)
66
67 def solve_problem3(dt=DT, dr=DR, t_end=None):
68     # 计算至阈值或终点
69     if t_end is None:
70         t_end = get_simulation_end(dt=dt)
71
72     # 5500 s后切换恒温边界
73     T_func, C_func = get_full_boundary_funcs()
74     t, r, T, C = solve_pde(
75         dt=dt,
76         dr=dr,
77         r_max=R_MAX,
78         t_end=t_end,
79         t_start=0.0,
80         initial_file=None,

```

```

80         initial_T=28.0,
81         initial_C=2.55,
82         T_air_func=T_func,
83         C_air_func=C_func,
84     )
85
86     max_C = np.max(C, axis=1)
87     hit = np.flatnonzero(max_C <= MOISTURE_LIMIT)
88     if len(hit) == 0:
89         return t, r, T, C, None, max_C
90
91     j = int(hit[0])
92     if j == 0:
93         drying_time = 0.0
94     else:
95         # 线性插值结束时刻
96         t0, t1 = t[j - 1], t[j]
97         c0, c1 = max_C[j - 1], max_C[j]
98         drying_time = float(
99             t0 + (MOISTURE_LIMIT - c0) * (t1 - t0) / (c1 - c0)
100         )
101     return t, r, T, C, drying_time, max_C
102
103
104 def _select_rows(t, times):
105     # 取最近时间行
106     rows = []
107     for target in times:
108         if target < t[0] - 1e-9 or target > t[-1] + 1e-9:
109             continue
110         rows.append(int(np.argmin(np.abs(t - target))))
111     return np.asarray(rows, dtype=int)
112
113
114 def verify_q2_checkpoint():
115     # 核对5500 s剖面
116     if not DEFAULT_INITIAL_FILE.exists():
117         raise FileNotFoundError(f"未找到问题二初始文件: {
118             DEFAULT_INITIAL_FILE}")
119     T_func, C_func = get_boundary_funcs(
120         path=OUTPUT_DIR / "output1.xlsx", t_start=0.0

```

```

121     _, r, T, C = solve_pde(
122         1.0,
123         DR,
124         r_max=R_MAX,
125         t_end=5500.0,
126         t_start=0.0,
127         initial_file=None,
128         initial_T=28.0,
129         initial_C=2.55,
130         T_air_func=T_func,
131         C_air_func=C_func,
132     )
133     _, T_ref, C_ref = load_initial_profile(
134         DEFAULT_INITIAL_FILE, r_max=R_MAX, Nr=len(r)
135     )
136     err_T = float(np.max(np.abs(T[-1] - T_ref)))
137     err_C = float(np.max(np.abs(C[-1] - C_ref)))
138     print(f"5500 s 剖面校验: T={err_T:.3e}, C={err_C:.3e}")
139     if err_T > 1e-4 or err_C > 1e-4:
140         raise AssertionError("第三问与问题二的 5500 s 剖面不一致")
141     return err_T, err_C
142
143
144 def save_problem3_results(
145     output_file=OUTPUT_DIR / "result3.xlsx",
146     summary_file=OUTPUT_DIR / "result3_summary.xlsx",
147 ):
148     # 保存完整表和摘要
149     t_raw, r, T_raw, C_raw, drying_time, max_C_raw =
150         solve_problem3()
151     if drying_time is None:
152         t, T, C, max_C = t_raw, T_raw, C_raw, max_C_raw
153         end_time = float(t[-1])
154         end_label = f"最大计算时长内未达到 {MOISTURE_LIMIT:g}"
155     else:
156         end_idx = int(np.flatnonzero(max_C_raw <=
157             MOISTURE_LIMIT)[0])
158         # 保留整步并追加结束行
159         t = t_raw[:end_idx]
160         T = T_raw[:end_idx]

```

```

160     C = C_raw[:end_idx]
161     max_C = max_C_raw[:end_idx]
162     if end_idx == 0:
163         frac = 0.0
164     else:
165         t_prev = t_raw[end_idx - 1]
166         frac = (drying_time - t_prev) / (t_raw[end_idx] -
            t_prev)
167     t = np.append(t, drying_time)
168     T_end = T_raw[max(end_idx - 1, 0)] + frac * (
169         T_raw[end_idx] - T_raw[max(end_idx - 1, 0)]
170     )
171     C_end = C_raw[max(end_idx - 1, 0)] + frac * (
172         C_raw[end_idx] - C_raw[max(end_idx - 1, 0)]
173     )
174     T = np.vstack([T, T_end])
175     C = np.vstack([C, C_end])
176     max_C = np.append(max_C, np.max(C_end))
177     end_time = drying_time
178     end_label = f"烘干结束: {drying_time / 3600.0:.4f} h
        (自 0 s) "
179
180     radii_cm = [f"{value * 100.0:.1f}" for value in r]
181     time_s = np.round(t, 2)
182     df_full = pd.DataFrame(np.round(C, 4), columns=radii_cm)
183     df_full.insert(0, "时间", time_s)
184
185     # 取6 h整数倍
186     regular_hours = np.arange(
187         6.0, np.floor(end_time / 21600.0) * 6.0 + 1e-12, 6.0
188     )
189     summary_idx = _select_rows(t, regular_hours * 3600.0)
190     target_radii = ["0.0", "0.5", "1.0", "1.5", "2.0"]
191     summary_rows = []
192     for idx in summary_idx:
193         row = {"时间/h": round(float(t[idx] / 3600.0), 4)}
194         for radius in target_radii:
195             radius_idx = int(round(float(radius) / (r[1] *
                100.0)))
196             row[radius] = round(float(C[idx, radius_idx]), 4)
197         summary_rows.append(row)
198

```

```

199     if drying_time is not None:
200         end_idx = int(np.flatnonzero(max_C_raw <=
                MOISTURE_LIMIT)[0])
201         if end_idx == 0:
202             C_end = C_raw[0]
203         else:
204             t0, t1 = t_raw[end_idx - 1], t_raw[end_idx]
205             frac = (drying_time - t0) / (t1 - t0)
206             C_end = C_raw[end_idx - 1] + frac * (C_raw[end_idx]
                - C_raw[end_idx - 1])
207         row = {"时间/h": round(float(drying_time / 3600.0), 4)}
208         for radius in target_radii:
209             radius_idx = int(round(float(radius) / (r[1] *
                100.0)))
210             row[radius] = float(round(C_end[radius_idx], 4))
211         summary_rows.append(row)
212
213     df_summary = pd.DataFrame(
214         summary_rows, columns=["时间/h"] + target_radii
215     )
216
217     output_file = Path(output_file)
218     summary_file = Path(summary_file)
219     if not output_file.is_absolute():
220         output_file = BASE_DIR / output_file
221     if not summary_file.is_absolute():
222         summary_file = BASE_DIR / summary_file
223     output_file.parent.mkdir(parents=True, exist_ok=True)
224     summary_file.parent.mkdir(parents=True, exist_ok=True)
225     df_full.to_excel(output_file, index=False)
226     df_summary.to_excel(summary_file, index=False)
227
228     print(f"问题三完整结果: {output_file}")
229     print(f"问题三摘要结果: {summary_file}")
230     print(end_label)
231     return output_file, summary_file, t, r, T, C, drying_time,
        max_C
232
233
234 def verify_problem3():
235     # 检查初值、阈值和范围
236     print("\n问题三数值检验")

```

```

237     verify_q2_checkpoint()
238     _, _, t, r, T, C, drying_time, max_C =
        save_problem3_results()
239
240     err_T = float(np.max(np.abs(T[0] - 28.0)))
241     err_C = float(np.max(np.abs(C[0] - 2.55)))
242     print(f"初值最大误差: T={err_T:.3e}, C={err_C:.3e}")
243
244     if drying_time is None:
245         print(f"最大计算时长内未达阈值, 末端 max(C)={max_C
            [-1]:.6f}")
246     else:
247         cross = int(np.searchsorted(t, drying_time))
248         before = max_C[max(cross - 1, 0)]
249         after = max_C[min(cross, len(max_C) - 1)]
250         print(f"烘干时间: {drying_time:.2f} s = {drying_time /
            3600.0:.4f} h")
251         print(f"阈值前 max(C)={before:.6f}")
252         print(f"阈值后 max(C)={after:.6f}")
253         if not (before > MOISTURE_LIMIT and after <=
            MOISTURE_LIMIT):
254             raise AssertionError("阈值跨越判定错误")
255
256     print(f"max(C) 是否单调不增: {bool(np.all(np.diff(max_C) <=
        1e-10)))}")
257     print(f"温度范围: {T.min():.6f}~{T.max():.6f} °C")
258     print(f"水分范围: {C.min():.6f}~{C.max():.6f} kg/kg")
259     print("标准: 初值误差 < 1e-10; max(C) 单调不增; 阈值正确跨
        越; 无 NaN 和负值。")
260
261     if err_T > 1e-10 or err_C > 1e-10:
262         raise AssertionError("问题三初值不一致")
263     if not np.all(np.isfinite(T)) or not np.all(np.isfinite(C))
        :
264         raise AssertionError("问题三出现 NaN 或无穷值")
265     if np.any(C < -1e-12):
266         raise AssertionError("问题三出现负水分浓度")
267     if not np.all(np.diff(max_C) <= 1e-10):
268         raise AssertionError("全域最大水分浓度不是单调不增")
269
270     return drying_time
271

```

```

272
273 def verify_time_step(t_end=21600.0):
274     # 检查时间步收敛
275     print("\n问题三时间步收敛")
276     dt_list = [60.0, 30.0, 15.0]
277     results = {dt: solve_problem3(dt=dt, dr=DR, t_end=t_end)
                for dt in dt_list}
278     common_t = np.arange(0.0, t_end + 1e-9, 3600.0)
279     max_c = {}
280     for dt, (t, _, _, C, _, _) in results.items():
281         idx = np rint(common_t / dt).astype(int)
282         max_c[dt] = np.max(C[idx, :], axis=1)
283     e1 = float(np.max(np.abs(max_c[60.0] - max_c[30.0])))
284     e2 = float(np.max(np.abs(max_c[30.0] - max_c[15.0])))
285     print(f"误差: 60/30 s={e1:.6g}, 30/15 s={e2:.6g}, 收敛比={
        e2 / e1:.4f}")
286     print("标准: 第二次误差更小, 收敛比接近 0.5")
287     return e1, e2
288
289
290 def verify_space_step(t_end=21600.0):
291     # 检查空间步收敛
292     print("\n问题三空间步长收敛")
293     dr_list = [0.001, 0.0005, 0.00025]
294     results = {dr: solve_problem3(dt=DT, dr=dr, t_end=t_end)
                for dr in dr_list}
295     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
296     final = {}
297     for dr, (_, r, _, C, _, _) in results.items():
298         final[dr] = np.interp(common_r, r, C[-1, :])
299     e1 = float(np.max(np.abs(final[0.001] - final[0.0005])))
300     e2 = float(np.max(np.abs(final[0.0005] - final[0.00025])))
301     print(f"误差: 1/0.5 mm={e1:.6g}, 0.5/0.25 mm={e2:.6g}, 收敛
        比={e2 / e1:.4f}")
302     print("标准: 第二次误差更小, 收敛比接近 0.25")
303     return e1, e2
304
305
306 def verify_conservation(t_end=21600.0):
307     # 检查水分守恒
308     print("\n问题三水分守恒")
309     T_func, C_func = get_full_boundary_funcs()

```

```

310     t, r, _, C = solve_pde(
311         DT,
312         DR,
313         r_max=R_MAX,
314         t_end=t_end,
315         t_start=0.0,
316         initial_file=None,
317         initial_T=28.0,
318         initial_C=2.55,
319         T_air_func=T_func,
320         C_air_func=C_func,
321     )
322     V, r_surf = _control_volumes(DR, len(r))
323     hm = 8e-7
324     area = 2.0 * np.pi * r_surf
325     M = np.sum(C * V, axis=1)
326     c_air_next = np.asarray(C_func(t[1:]), dtype=float)
327     flux = hm * area * (c_air_next - C[1:, -1])
328     flux_integral = np.concatenate([[0.0], np.cumsum(flux * DT)
329                                     ])
329     residual = abs((M[-1] - M[0]) - flux_integral[-1]) / max(
330         abs(M[-1] - M[0]), 1e-12
331     )
332     print(f"最终相对残差: {residual:.4%}")
333     print("标准: 水分守恒残差小于 1%")
334     if residual >= 0.01:
335         raise AssertionError("问题三水分守恒未通过")
336     return residual
337
338
339 def plot_problem3_results(t, r, C, drying_time):
340     # 结果图统一由 plot_condensed.py 生成
341     return None
342
343
344 if __name__ == "__main__":
345     result = save_problem3_results()
346     verify_problem3()
347     verify_time_step()
348     verify_space_step()
349     verify_conservation()

```

q4.py

```

1  # 问题四变半径模型
2
3  from pathlib import Path
4
5  import numpy as np
6  import pandas as pd
7  from scipy.interpolate import PchipInterpolator
8
9  from q3 import get_full_boundary_funcs
10
11
12  BASE_DIR = Path(__file__).resolve().parent
13  INPUT_DIR = BASE_DIR / "input_excel"
14  OUTPUT_DIR = BASE_DIR / "output_excel"
15  RADIUS_FILE = INPUT_DIR / "附件2.xlsx"
16  if not RADIUS_FILE.exists():
17      RADIUS_FILE = BASE_DIR / "附件" / "附件2.xlsx"
18
19  MOISTURE_LIMIT = 0.15
20  DT = 60.0
21  DX = 0.05
22  R0 = 0.02
23  MAX_TIME = 259200.0
24  R_MIN = 0.05
25  R_MAX = 10.0
26  T_MIN = -50.0
27  T_MAX = 250.0
28
29
30  def load_radius_data(path=RADIUS_FILE):
31      # 读取半径数据
32      path = Path(path)
33      if not path.exists():
34          raise FileNotFoundError(f"未找到半径文件: {path}")
35      df = pd.read_excel(path)
36      if df.shape[1] < 2:
37          raise ValueError("附件2至少需要时间和半径两列")
38      t = df.iloc[:, 0].to_numpy(dtype=float)
39      radius_cm = df.iloc[:, 1].to_numpy(dtype=float)
40      if not (np.all(np.isfinite(t)) and np.all(np.isfinite(
41          radius_cm))):
42          raise ValueError("附件2含有NaN或无穷值")

```

```

42     if np.any(np.diff(t) <= 0) or np.any(radius_cm <= 0):
43         raise ValueError("附件2时间须递增且半径须为正")
44     if t[0] > 1e-12 or t[-1] < MAX_TIME - 1e-12:
45         raise ValueError("附件2必须覆盖0~259200 s")
46     if np.any(np.diff(radius_cm) > 1e-10):
47         raise ValueError("附件2半径应单调不减")
48     return t, radius_cm / 100.0
49
50
51 def interpolate_radius(dt=DT, t_end=MAX_TIME, path=RADIUS_FILE)
52 :
53     # PCHIP插值
54     t_raw, r_raw = load_radius_data(path)
55     if dt <= 0 or t_end < 0:
56         raise ValueError("时间参数必须为正")
57     n = int(round(t_end / dt))
58     t = np.arange(n + 1, dtype=float) * dt
59     if t[-1] > t_raw[-1] + 1e-9:
60         raise ValueError("插值终点超过附件2范围")
61     pchip = PchipInterpolator(t_raw, r_raw)
62     r = np.asarray(pchip(t), dtype=float)
63     # 修正浮点误差
64     r = np.minimum.accumulate(r)
65     if np.any(r <= 0):
66         raise ValueError("插值后出现非正半径")
67     return t, r, t_raw, r_raw
68
69 def save_radius_interpolation():
70     # 保存半径插值
71     t, r, t_raw, r_raw = interpolate_radius()
72     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
73     pd.DataFrame({"时间/s": t.astype(int), "半径/m": r, "半径/
74                  cm": r * 100.0}).to_excel(
75         OUTPUT_DIR / "q4_radius_interp.xlsx", index=False
76     )
77     return t, r
78
79 # 附录4物性公式
80 def rho_of_C(C):
81     return 760.0 + 90.0 * C

```

```

82
83
84 def cp_of_C(C):
85     C_safe = np.maximum(C, 1e-8)
86     return 1850.0 + 2150.0 * C_safe / (C_safe + 1.0)
87
88
89 def k_of_C(C):
90     C_safe = np.maximum(C, 1e-8)
91     return 0.12 + 0.20 * C_safe / (C_safe + 1.0)
92
93
94 def D_of_CT(C, T):
95     C_safe = np.clip(C, 0.05, 10.0)
96     T_K = np.clip(T + 273.15, 273.15, 373.15)
97     D = 4.2e-4 * np.exp(-0.30 / C_safe) * np.exp(-3850.0 / T_K)
98     return np.clip(D, 1e-14, 1e-3)
99
100
101 def solve_tridiag(a, b, c, d):
102     # 追赶法
103     n = len(b)
104     if n == 0:
105         return np.empty(0)
106     cp = np.zeros(n)
107     dp = np.zeros(n)
108     if abs(b[0]) < 1e-14:
109         raise ZeroDivisionError("三对角矩阵首行奇异")
110     cp[0] = c[0] / b[0]
111     dp[0] = d[0] / b[0]
112     for i in range(1, n):
113         den = b[i] - a[i] * cp[i - 1]
114         if abs(den) < 1e-14:
115             raise ZeroDivisionError(f"三对角矩阵第{i}行奇异")
116         if i < n - 1:
117             cp[i] = c[i] / den
118             dp[i] = (d[i] - a[i] * dp[i - 1]) / den
119     x = np.zeros(n)
120     x[-1] = dp[-1]
121     for i in range(n - 2, -1, -1):
122         x[i] = dp[i] - cp[i] * x[i + 1]
123     return x

```

```

124
125
126 def _assemble_scalar(old, coeff, storage, radius, dt, dx,
127                       boundary_value, boundary_coeff):
128     # 组装隐式方程
129     nr = len(old)
130     x = np.linspace(0.0, 1.0, nr)
131     w = np.zeros(nr)
132     w[0] = (dx / 2.0) ** 2
133     w[1:-1] = 2.0 * x[1:-1] * dx
134     w[-1] = 1.0 - (1.0 - dx / 2.0) ** 2
135     a = np.zeros(nr)
136     b = np.ones(nr)
137     c = np.zeros(nr)
138     d = old.copy()
139     # 内部扩散通量
140     face_x = (np.arange(nr - 1) + 0.5) * dx
141     for f in range(nr - 1):
142         xi_face = face_x[f]
143         left, right = f, f + 1
144         # 相邻控制体共用通量
145         g_left = 2.0 * dt * xi_face * coeff[f] / (radius**2 *
146             storage[left] * w[left] * dx)
147         g_right = 2.0 * dt * xi_face * coeff[f] / (radius**2 *
148             storage[right] * w[right] * dx)
149         c[left] -= g_left
150         b[left] += g_left
151         a[right] -= g_right
152         b[right] += g_right
153     # 表面对流项
154     g_conv = 2.0 * dt * boundary_coeff / (radius * storage[-1]
155         * w[-1])
156     b[-1] += g_conv
157     d[-1] += g_conv * boundary_value
158     return a, b, c, d
159
160 def solve_problem4(dt=DT, dx=DX, t_end=MAX_TIME,
161                   stop_at_threshold=True):
162     # 求解变半径全过程
163     if dt <= 0 or dx <= 0 or t_end < 0:
164         raise ValueError("时间和空间步长必须为正")

```

```

162     n_interval = int(round(1.0 / dx))
163     if not np.isclose(n_interval * dx, 1.0, atol=1e-12, rtol
164                       =0.0):
165         raise ValueError("dx 必须能整除无量纲区间 [0,1]")
166     nr = n_interval + 1
167     x = np.linspace(0.0, 1.0, nr)
168     t, radius, _, _ = interpolate_radius(dt=dt, t_end=t_end)
169     T_air_func, C_air_func = get_full_boundary_funcs()
170     T = np.zeros((len(t), nr))
171     C = np.zeros((len(t), nr))
172     T[0, :] = 28.0
173     C[0, :] = 2.55
174     max_C = np.zeros(len(t))
175     max_C[0] = np.max(C[0])
176     hit = None
177
178     for n in range(len(t) - 1):
179         Rnext = radius[n + 1]
180         T_air = float(T_air_func(t[n + 1]))
181         C_air = float(C_air_func(t[n + 1]))
182         C_mid = 0.5 * (C[n, :-1] + C[n, 1:])
183         T_mid = 0.5 * (T[n, :-1] + T[n, 1:])
184         k_face = k_of_C(C_mid)
185         D_face = D_of_CT(C_mid, T_mid)
186         H = rho_of_C(C[n]) * cp_of_C(C[n])
187
188         aT, bT, cT, dT = _assemble_scalar(
189             T[n], k_face, H, Rnext, dt, dx, T_air, 25.0,
190             # 温度存储量
191             T[n + 1] = solve_tridiag(aT, bT, cT, dT)
192
193         aC, bC, cC, dC = _assemble_scalar(
194             C[n], D_face, np.ones(nr), Rnext, dt, dx, C_air,
195             8.0e-7,
196             C[n + 1] = solve_tridiag(aC, bC, cC, dC)
197         T[n + 1] = np.clip(T[n + 1], T_MIN, T_MAX)
198         C[n + 1] = np.clip(C[n + 1], 0.0, R_MAX)
199         max_C[n + 1] = np.max(C[n + 1])
200         if hit is None and max_C[n + 1] <= MOISTURE_LIMIT:
201             hit = n + 1

```

```

202         if stop_at_threshold:
203             break
204
205     n_used = len(t) if (hit is None or not stop_at_threshold)
206         else hit + 1
207     t = t[:n_used]
208     radius = radius[:n_used]
209     T = T[:n_used]
210     C = C[:n_used]
211     max_C = max_C[:n_used]
212     drying_time = None
213     if hit is not None:
214         if hit == 0:
215             drying_time = 0.0
216         else:
217             drying_time = float(
218                 t[hit - 1] + (MOISTURE_LIMIT - max_C[hit - 1])
219                 * (t[hit] - t[hit - 1]) / (max_C[hit] - max_C[
220                     hit - 1])
221             )
222     return t, radius, x, T, C, max_C, drying_time
223
224 def _append_interpolated_end(t, radius, T, C, max_C,
225     drying_time):
226     # 追加插值结束行
227     if drying_time is None or drying_time >= t[-1] - 1e-12:
228         return t, radius, T, C, max_C
229     j = len(t) - 1
230     t0, t1 = t[j - 1], t[j]
231     f = (drying_time - t0) / (t1 - t0)
232     t = np.append(t[:-1], drying_time)
233     radius = np.append(radius[:-1], radius[j - 1] + f * (radius
234         [j] - radius[j - 1]))
235     T = np.vstack([T[:-1], T[j - 1] + f * (T[j] - T[j - 1])])
236     C = np.vstack([C[:-1], C[j - 1] + f * (C[j] - C[j - 1])])
237     max_C = np.append(max_C[:-1], np.max(C[-1]))
238     return t, radius, T, C, max_C
239
240 def _physical_table(t, radius, x, C):
241     # 输出物理半径网格

```

```

240 target_cm = np.round(np.arange(0.0, 2.0 + 1e-12, 0.1), 1)
241 rows = []
242 for n, time in enumerate(t):
243     row = {"时间/s": round(float(time), 2)}
244     r_cm = radius[n] * 100.0
245     r_nodes = x * radius[n] * 100.0
246     for p in target_cm:
247         key = f"{p:.1f}"
248         row[key] = round(float(np.interp(p, r_nodes, C[n])),
249                             , 4) if p <= r_cm + 1e-10 else None
249     row["药材表面"] = round(float(C[n, -1]), 4)
250     row["半径/cm"] = round(float(r_cm), 4)
251     rows.append(row)
252 return pd.DataFrame(rows, columns=["时间/s"] + [f"{p:.1f}"
253     for p in target_cm] + ["药材表面", "半径/cm"])
254
255 def _summary_table(t, radius, x, C, drying_time):
256     # 生成表6摘要
257     regular = np.arange(6.0, np.floor(t[-1] / 21600.0) * 6.0 +
258         1e-12, 6.0) * 3600.0
259     targets = list(regular)
260     if drying_time is not None and (not targets or drying_time
261         > targets[-1] + 1e-8):
262         targets.append(drying_time)
263     target_cm = [round(v, 1) for v in np.arange(0.0, 2.0 + 1e
264         -12, 0.5)]
265     rows = []
266     for target in targets:
267         idx = int(np.argmin(np.abs(t - target)))
268         r_cm = radius[idx] * 100.0
269         r_nodes = x * radius[idx] * 100.0
270         row = {"时间/h": round(float(t[idx] / 3600.0), 4), "半
271             径/cm": round(float(r_cm), 4)}
272         for p in target_cm:
273             row[f"{p:.1f}"] = round(float(np.interp(p, r_nodes,
274                 C[idx])), 4) if p <= r_cm + 1e-10 else None
275         row["药材表面"] = round(float(C[idx, -1]), 4)
276         rows.append(row)
277     cols = ["时间/h"] + [f"{p:.1f}" for p in target_cm] + ["药
278         材表面", "半径/cm"]
279     return pd.DataFrame(rows, columns=cols)

```

```

274
275
276 def save_results():
277     # 保存第四问结果
278     t, radius, x, T, C, max_C, drying_time = solve_problem4()
279     t, radius, T, C, max_C = _append_interpolated_end(t, radius
        , T, C, max_C, drying_time)
280     full = _physical_table(t, radius, x, C)
281     summary = _summary_table(t, radius, x, C, drying_time)
282     normalized = pd.DataFrame(np.round(C, 4), columns=[f"xi={v
        :.2f}" for v in x])
283     normalized.insert(0, "时间/s", np.round(t, 2))
284     normalized["半径/cm"] = np.round(radius * 100.0, 4)
285     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
286     with pd.ExcelWriter(OUTPUT_DIR / "result4.xlsx") as writer:
287         full.to_excel(writer, sheet_name="水分浓度", index=
            False)
288         summary.to_excel(writer, sheet_name="表6摘要", index=
            False)
289         normalized.to_excel(writer, sheet_name="无量纲网格",
            index=False)
290         pd.DataFrame({"时间/s": np.round(t, 2), "半径/cm": np.
            round(radius * 100.0, 6)}).to_excel(
291             writer, sheet_name="半径插值", index=False
292         )
293     # 保存表6格式
294     q4_1 = summary[["时间/h", "0.0", "0.5", "1.0", "1.5", "药材
        表面"]].copy()
295     q4_1.to_excel(OUTPUT_DIR / "q4_1.xlsx", index=False)
296     _plot_results(t, radius, x, C, max_C, drying_time)
297     print(f"第四问结果已保存: {OUTPUT_DIR / 'result4.xlsx'}")
298     print(f"表6结果已保存: {OUTPUT_DIR / 'q4_1.xlsx'}")
299     if drying_time is None:
300         print("最大计算时长内未达到水分阈值")
301     else:
302         print(f"第四问烘干时间: {drying_time:.4f} s = {
            drying_time / 3600.0:.5f} h")
303     return t, radius, x, T, C, max_C, drying_time
304
305
306 def _plot_results(t, radius, x, C, max_C, drying_time):
307     # 结果图统一由 plot_condensed.py 生成

```

```

308     return None
309
310
311 def validate_q4():
312     # 校验第四问
313     result = solve_problem4(stop_at_threshold=False)
314     t, radius, x, T, C, max_C, drying_time = result
315     if not (np.all(np.isfinite(T)) and np.all(np.isfinite(C))
316             and np.all(C >= -1e-12)):
317         raise AssertionError("第四问出现非有限值或负水分")
318     if np.any(np.diff(radius) > 1e-12):
319         raise AssertionError("第四问半径未保持单调不增")
320     # 检查半径和水分浓度趋势
321     stable_idx = np.searchsorted(t, 5500.0)
322     if np.any(np.diff(max_C[stable_idx:]) > 1e-7):
323         raise AssertionError("第四问恒温段全域最大水分浓度不单调")
324     if drying_time is None:
325         hit = np.flatnonzero(max_C <= MOISTURE_LIMIT)
326         if len(hit):
327             raise AssertionError("第四问阈值时间记录异常")
328     # 检查材料坐标守恒
329     dt = float(t[1] - t[0]) if len(t) > 1 else DT
330     w = np.zeros(len(x))
331     w[0] = (DX / 2.0) ** 2
332     w[1:-1] = 2.0 * x[1:-1] * DX
333     w[-1] = 1.0 - (1.0 - DX / 2.0) ** 2
334     _, C_air = get_full_boundary_funcs()
335     air = np.asarray(C_air(t[1:]), dtype=float)
336     flux = 2.0 * dt * 8.0e-7 / radius[1:] * (air - C[1:, -1])
337     mass_res = float(np.max(np.abs(np.sum((C[1:] - C[:-1]) * w,
338             axis=1) - flux)))
339     if mass_res > 1e-10:
340         raise AssertionError(f"第四问水分守恒残差过大: {
341             mass_res:.3e}")
342     return {"time_end_s": float(t[-1]), "radius_start_cm":
343             float(radius[0] * 100),
344             "radius_end_cm": float(radius[-1] * 100), "C_min":
345             float(C.min()),
346             "C_max": float(C.max()), "drying_time_s":
347             drying_time,
348             "moisture_balance_abs": mass_res}

```

```

343
344
345 if __name__ == "__main__":
346     save_radius_interpolation()
347     save_results()

```

validate_all.py

```

1  # 四问独立重算和交叉检验
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import pandas as pd
11
12 from q1_3 import get_boundary_funcs as get_q1_boundary
13 from q1_3 import solve_pde as solve_q1
14 from q2_3 import (
15     DEFAULT_INITIAL_FILE,
16     INITIAL_TIME,
17     OUTPUT_DIR,
18     get_boundary_funcs as get_q2_boundary,
19     load_initial_profile,
20     solve_pde as solve_q2,
21 )
22 from q3 import MOISTURE_LIMIT, solve_problem3
23 import q4
24
25
26 BASE_DIR = Path(__file__).resolve().parent
27 PIC_DIR = BASE_DIR / "pic"
28 REPORT_FILE = OUTPUT_DIR / "validation_report.json"
29 R_MAX = 0.02
30
31 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei",
32     "Arial Unicode MS"]
33 plt.rcParams["axes.unicode_minus"] = False
34

```

```

35 def control_volumes(dr: float, nr: int):
36     # 计算控制体体积
37     v = np.zeros(nr)
38     v[0] = np.pi * (dr / 2.0) ** 2
39     for i in range(1, nr - 1):
40         v[i] = 2.0 * np.pi * (i * dr) * dr
41     r_surface = (nr - 1) * dr
42     v[-1] = np.pi * (r_surface**2 - (r_surface - dr / 2.0) **
43                     2)
44     return v, 2.0 * np.pi * r_surface
45
46 def read_sheet(path: Path, sheet_index: int = 0) -> pd.
    DataFrame:
47     book = pd.ExcelFile(path)
48     return pd.read_excel(path, sheet_name=book.sheet_names[
49         sheet_index])
50
51 def add_check(report: dict, name: str, passed: bool, value=None
52 , criterion: str = ""):
53     item = {"passed": bool(passed), "criterion": criterion}
54     if value is not None:
55         item["value"] = value
56     report["checks"][name] = item
57     if not passed:
58         raise AssertionError(f"验证未通过: {name}; value={value
59                               }; criterion={criterion}")
60
61 def validate_file_structure(report: dict):
62     # 检查文件结构
63     expected = {
64         "result1.xlsx": (2, 1801, 0.0, 1800.0, 1.0),
65         "result2.xlsx": (2, 10801, 0.0, 10800.0, 1.0),
66     }
67     for name, (n_sheet, n_rows, t0, t1, step) in expected.items
68     ():
69         path = OUTPUT_DIR / name
70         book = pd.ExcelFile(path)
71         ok_sheets = len(book.sheet_names) == n_sheet
72         add_check(report, f"{name}_工作表", ok_sheets, book.

```

```

    sheet_names, f"工作表数={n_sheet}")
71     for j in range(n_sheet):
72         df = pd.read_excel(path, sheet_name=book.
            sheet_names[j])
73         times = df.iloc[:, 0].to_numpy(dtype=float)
74         values = df.iloc[:, 1:].to_numpy(dtype=float)
75         radii = list(df.columns[1:])
76         ok = (
77             df.shape == (n_rows, 22)
78             and np.isclose(times[0], t0)
79             and np.isclose(times[-1], t1)
80             and np.allclose(np.diff(times), step)
81             and radii == [f"{i / 10:.1f}" for i in range
                (21)]
82             and np.all(np.isfinite(values))
83             and np.allclose(values, np.round(values, 4))
84         )
85         add_check(
86             report,
87             f"{name}_表{j + 1}_结构",
88             ok,
89             {"shape": list(df.shape), "time": [float(times
                [0]), float(times[-1])]},
90             "22列; 时间连续; 半径0.0~2.0 cm; 四位小数; 无
                NaN",
91         )
92
93     path = OUTPUT_DIR / "result3.xlsx"
94     df = read_sheet(path)
95     times = df.iloc[:, 0].to_numpy(dtype=float)
96     values = df.iloc[:, 1:].to_numpy(dtype=float)
97     ok = (
98         df.shape == (3512, 22)
99         and np.isclose(times[0], 0.0)
100        and np.isclose(times[-1], 210634.74, atol=0.02)
101        and np.all(np.diff(times[:-1]) == 60.0)
102        and np.all(np.isfinite(values))
103        and np.allclose(values, np.round(values, 4))
104    )
105    add_check(
106        report,
107        "result3_结构",

```

```

108         ok,
109         {"shape": list(df.shape), "first_time_s": float(times
110             [0]), "last_time_s": float(times[-1])},
111         "0~烘干结束; 60 s网格并追加插值结束行; 无NaN; 四位小数"
112     )
113
114     summary = read_sheet(OUTPUT_DIR / "result3_summary.xlsx")
115     summary_t = summary.iloc[:, 0].to_numpy(dtype=float)
116     ok = (
117         summary.shape == (10, 6)
118         and np.allclose(summary_t[:-1], np.arange(6.0, 55.0,
119             6.0))
120         and np.isclose(summary_t[-1], 210634.74 / 3600.0, atol
121             =1e-4)
122         and np.all(np.isfinite(summary.iloc[:, 1:].to_numpy(
123             dtype=float)))
124     )
125     add_check(report, "result3_摘要结构", ok, summary_t.tolist
126         (), "每6 h及结束时刻")
127
128     profile = read_sheet(OUTPUT_DIR / "output3.xlsx")
129     ok = profile.shape == (21, 3) and np.all(np.isfinite(
130         profile.iloc[:, 1:].to_numpy(dtype=float)));
131     add_check(report, "output3_结构", ok, list(profile.shape),
132         "21个径向节点, 温度和水分均有限")
133
134     path = OUTPUT_DIR / "result4.xlsx"
135     book = pd.ExcelFile(path)
136     full = pd.read_excel(path, sheet_name=book.sheet_names[0])
137     normalized = pd.read_excel(path, sheet_name=book.
138         sheet_names[2])
139     radius = pd.read_excel(path, sheet_name=book.sheet_names
140         [3])
141     times = full.iloc[:, 0].to_numpy(dtype=float)
142     radii = full.iloc[:, -1].to_numpy(dtype=float)
143     values = full.iloc[:, 1:22].to_numpy(dtype=float)
144     physical_ok = True
145     target_cm = np.arange(0.0, 2.0 + 1e-12, 0.1)
146     for i in range(len(full)):
147         inside = target_cm <= radii[i] + 1e-8
148         physical_ok &= np.all(np.isfinite(values[i, inside]))

```

```

140         and np.all(~np.isfinite(values[i, ~inside]))
141     ok = (
142         len(book.sheet_names) == 4
143         and full.shape[1] == 24
144         and normalized.shape[1] == 23
145         and radius.shape[1] == 2
146         and np.isclose(times[0], 0.0)
147         and np.isclose(times[-1], 184815.74, atol=0.02)
148         and np.all(np.diff(times) > 0)
149         and physical_ok
150         and np.all(np.isfinite(normalized.iloc[:, 1:].to_numpy(
151             dtype=float)))
152         and np.all(np.isfinite(radius.iloc[:, 1:].to_numpy(
153             dtype=float)))
154     )
155     add_check(report, "result4_结构", ok,
156               {"sheets": book.sheet_names, "shape": list(full.
157                   shape),
158                "first_time_s": float(times[0]), "last_time_s":
159                    float(times[-1])},
160               "变半径水分表、表6摘要、无量纲网格和半径插值均存
161               在；域外物理位置留空")
162
163 def q1_recompute_and_balance(report: dict):
164     # 重算问题一并检查守恒
165     t_func, c_func = get_q1_boundary()
166     t, r, temp, moist = solve_q1(1.0, 0.001, t_end=1800.0,
167                                   T_air_func=t_func, C_air_func=c_func)
168     book = pd.ExcelFile(OUTPUT_DIR / "result1.xlsx")
169     stored_t = read_sheet(OUTPUT_DIR / "result1.xlsx", 0).iloc
170        [:, 1:].to_numpy(dtype=float)
171     stored_c = read_sheet(OUTPUT_DIR / "result1.xlsx", 1).iloc
172        [:, 1:].to_numpy(dtype=float)
173     err_t = float(np.max(np.abs(np.round(temp, 4) - stored_t)))
174     err_c = float(np.max(np.abs(np.round(moist, 4) - stored_c))
175                 )
176     add_check(report, "问题一_独立重算", err_t <= 1e-12 and
177               err_c <= 1e-12,
178               {"T_max_abs": err_t, "C_max_abs": err_c}, "重算结
179               果与Excel四位小数结果一致")
180

```

```

170     v, area = control_volumes(0.001, len(r))
171     rho, cp, h, hm = 820.0, 2600.0, 25.0, 8e-7
172     air_t = np.asarray(t_func(t), dtype=float)
173     air_c = np.asarray(c_func(t), dtype=float)
174     q_step = h * area * (air_t[1:] - temp[:-1, -1])
175     m_step = hm * area * (air_c[1:] - moist[:-1, -1])
176     storage_q = np.sum(rho * cp * (temp[1:] - temp[:-1])) * v,
        axis=1)
177     storage_m = np.sum((moist[1:] - moist[:-1]) * v, axis=1)
178     q_res = abs(np.sum(storage_q) - np.sum(q_step)) / max(abs(
        np.sum(storage_q)), 1e-12)
179     m_res = abs(np.sum(storage_m) - np.sum(m_step)) / max(abs(
        np.sum(storage_m)), 1e-12)
180     add_check(report, "问题一_守恒", q_res < 0.01 and m_res <
        0.01,
181             {"energy_relative": q_res, "moisture_relative":
        m_res}, "两项相对残差<1%")
182     return {"t": t, "r": r, "T": temp, "C": moist, "q_res":
        q_res, "m_res": m_res}
183
184
185 def q2_recompute_and_balance(report: dict):
186     # 重算问题二并检查断点
187     t_func, c_func = get_q2_boundary(t_start=INITIAL_TIME)
188     t, r, temp, moist = solve_q2(
189         1.0, 0.001, t_end=10800.0, t_start=INITIAL_TIME,
190         initial_file=DEFAULT_INITIAL_FILE, T_air_func=t_func,
191         C_air_func=c_func,
192     )
193     stored_t = read_sheet(OUTPUT_DIR / "result2.xlsx", 0).iloc
       [:, 1:].to_numpy(dtype=float)
194     stored_c = read_sheet(OUTPUT_DIR / "result2.xlsx", 1).iloc
       [:, 1:].to_numpy(dtype=float)
195     err_t = float(np.max(np.abs(np.round(temp, 4) - stored_t)))
196     err_c = float(np.max(np.abs(np.round(moist, 4) - stored_c))
        )
197     add_check(report, "问题二_独立重算", err_t <= 1e-12 and
        err_c <= 1e-12,
198             {"T_max_abs": err_t, "C_max_abs": err_c}, "重算结
        果与Excel四位小数结果一致")
199     _, t0, c0 = load_initial_profile(DEFAULT_INITIAL_FILE,

```

```

    r_max=R_MAX, Nr=len(r))
200 add_check(report, "问题二_初始剖面", np.max(abs(temp[0] -
    t0)) <= 1e-12 and np.max(abs(moist[0] - c0)) <= 1e-12,
201     {"T_max_abs": float(np.max(abs(temp[0] - t0))), "
        C_max_abs": float(np.max(abs(moist[0] - c0)))
        },
202     "t=0行等于output3.xlsx")
203
204 v, area = control_volumes(0.001, len(r))
205 h, hm = 25.0, 8e-7
206 air_t = np.asarray(t_func(INITIAL_TIME + t), dtype=float)
207 air_c = np.asarray(c_func(INITIAL_TIME + t), dtype=float)
208 rho = 650.0 + 128.0 * moist[:-1]
209 cp = 1450.0 + 2736.0 * (moist[:-1] / (moist[:-1] + 1.0))
210 q_step = h * area * (air_t[1:] - temp[1:, -1])
211 m_step = hm * area * (air_c[1:] - moist[1:, -1])
212 storage_q = np.sum(rho * cp * (temp[1:] - temp[:-1]) * v,
    axis=1)
213 storage_m = np.sum((moist[1:] - moist[:-1]) * v, axis=1)
214 q_res = abs(np.sum(storage_q) - np.sum(q_step)) / max(abs(
    np.sum(storage_q)), 1e-12)
215 m_res = abs(np.sum(storage_m) - np.sum(m_step)) / max(abs(
    np.sum(storage_m)), 1e-12)
216 finite = bool(np.all(np.isfinite(temp)) and np.all(np.
    isfinite(moist)))
217 physical = finite and bool(np.all(moist >= -1e-12)) and
    bool(np.all(temp > -50.0)) and bool(np.all(temp < 250.0))
218 add_check(report, "问题二_守恒", q_res < 0.01 and m_res <
    0.01,
219     {"energy_relative": q_res, "moisture_relative":
        m_res}, "两项相对残差<1%")
220 add_check(report, "问题二_物理范围", physical,
221     {"T_min": float(temp.min()), "T_max": float(temp.
        max()), "C_min": float(moist.min()), "C_max":
        float(moist.max())},
222     "有限值、C>=0且未触及截断")
223 return {"t": t, "r": r, "T": temp, "C": moist, "q_res":
    q_res, "m_res": m_res}
224
225
226 def convergence_q1(report: dict):

```

```

227     tf, cf = get_q1_boundary()
228     dt_list = [1.0, 0.5, 0.25]
229     sol = {dt: solve_q1(dt, 0.001, t_end=1800.0, T_air_func=tf,
        C_air_func=cf) for dt in dt_list}
230     common_t = np.arange(0.0, 1800.0 + 1e-9, 100.0)
231     vals = {dt: (x[2][np rint(common_t / dt).astype(int)], x
        [3][np rint(common_t / dt).astype(int)]) for dt, x in
        sol.items()}
232     e_t = [float(np.max(abs(vals[1.0][0] - vals[0.5][0]))),
        float(np.max(abs(vals[0.5][0] - vals[0.25][0])))]
233     e_c = [float(np.max(abs(vals[1.0][1] - vals[0.5][1]))),
        float(np.max(abs(vals[0.5][1] - vals[0.25][1])))]
234     sf = {"dt": [0.5, 0.25], "T_error": e_t, "C_error": e_c, "
        T_ratio": e_t[1] / e_t[0], "C_ratio": e_c[1] / e_c[0]}
235     add_check(report, "问题一_时间收敛", 0.35 < sf["T_ratio"] <
        0.65 and 0.35 < sf["C_ratio"] < 0.65 and e_t[1] < e_t
        [0] and e_c[1] < e_c[0], sf, "误差递减, 收敛比接近0.5")
236
237     dr_list = [0.001, 0.0005, 0.00025]
238     sol = {dr: solve_q1(0.05, dr, t_end=1800.0, T_air_func=tf,
        C_air_func=cf) for dr in dr_list}
239     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
240     final = {dr: (np.interp(common_r, x[1], x[2][-1]), np.
        interp(common_r, x[1], x[3][-1])) for dr, x in sol.items
        ()}
241     e_t = [float(np.max(abs(final[0.001][0] - final[0.0005][0])
        )), float(np.max(abs(final[0.0005][0] - final
        [0.00025][0])))]
242     e_c = [float(np.max(abs(final[0.001][1] - final[0.0005][1])
        )), float(np.max(abs(final[0.0005][1] - final
        [0.00025][1])))]
243     sf2 = {"dr": [0.0005, 0.00025], "T_error": e_t, "C_error":
        e_c, "T_ratio": e_t[1] / e_t[0], "C_ratio": e_c[1] / e_c
        [0]}
244     add_check(report, "问题一_空间收敛", 0.15 < sf2["T_ratio"]
        < 0.35 and 0.15 < sf2["C_ratio"] < 0.35 and e_t[1] < e_t
        [0] and e_c[1] < e_c[0], sf2, "误差递减, 收敛比接近0.25"
        )
245     return {"time": sf, "space": sf2}
246
247
248 def convergence_q2(report: dict):

```

```

249 tf, cf = get_q2_boundary(t_start=INITIAL_TIME)
250 dt_list = [1.0, 0.5, 0.25]
251 sol = {dt: solve_q2(dt, 0.001, t_end=1800.0, t_start=
    INITIAL_TIME, initial_file=DEFAULT_INITIAL_FILE,
    T_air_func=tf, C_air_func=cf) for dt in dt_list}
252 common_t = np.arange(0.0, 1800.0 + 1e-9, 100.0)
253 vals = {dt: (x[2][np rint(common_t / dt).astype(int)], x
    [3][np rint(common_t / dt).astype(int)]) for dt, x in
    sol.items()}
254 e_t = [float(np.max(abs(vals[1.0][0] - vals[0.5][0])),
    float(np.max(abs(vals[0.5][0] - vals[0.25][0])))]
255 e_c = [float(np.max(abs(vals[1.0][1] - vals[0.5][1])),
    float(np.max(abs(vals[0.5][1] - vals[0.25][1])))]
256 sf = {"dt": [0.5, 0.25], "T_error": e_t, "C_error": e_c, "
    T_ratio": e_t[1] / e_t[0], "C_ratio": e_c[1] / e_c[0]}
257 add_check(report, "问题二_时间收敛", 0.35 < sf["T_ratio"] <
    0.65 and 0.35 < sf["C_ratio"] < 0.65 and e_t[1] < e_t
    [0] and e_c[1] < e_c[0], sf, "误差递减, 收敛比接近0.5")
258
259 dr_list = [0.001, 0.0005, 0.00025]
260 sol = {dr: solve_q2(1.0, dr, t_end=1800.0, t_start=
    INITIAL_TIME, initial_file=DEFAULT_INITIAL_FILE,
    T_air_func=tf, C_air_func=cf) for dr in dr_list}
261 common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
262 final = {dr: (np.interp(common_r, x[1], x[2][-1]), np.
    interp(common_r, x[1], x[3][-1])) for dr, x in sol.items
    ()}
263 e_t = [float(np.max(abs(final[0.001][0] - final[0.0005][0])
    )), float(np.max(abs(final[0.0005][0] - final
    [0.00025][0])))]
264 e_c = [float(np.max(abs(final[0.001][1] - final[0.0005][1])
    )), float(np.max(abs(final[0.0005][1] - final
    [0.00025][1])))]
265 sf2 = {"dr": [0.0005, 0.00025], "T_error": e_t, "C_error":
    e_c, "T_ratio": e_t[1] / e_t[0], "C_ratio": e_c[1] / e_c
    [0]}
266 add_check(report, "问题二_空间收敛", 0.15 < sf2["T_ratio"]
    < 0.35 and 0.15 < sf2["C_ratio"] < 0.35 and e_t[1] < e_t
    [0] and e_c[1] < e_c[0], sf2, "误差递减, 收敛比接近0.25"
    )
267 return {"time": sf, "space": sf2}
268

```

```

269
270 def convergence_q3(report: dict):
271     dt_list = [60.0, 30.0, 15.0]
272     sol = {dt: solve_problem3(dt=dt, dr=0.001, t_end=21600.0)
            for dt in dt_list}
273     common_t = np.arange(0.0, 21600.0 + 1e-9, 3600.0)
274     maxc = {dt: np.max(x[3][np rint(common_t / dt).astype(int)
            ], axis=1) for dt, x in sol.items()}
275     e_t = [float(np.max(abs(maxc[60.0] - maxc[30.0]))), float(
            np.max(abs(maxc[30.0] - maxc[15.0])))]
276     sf = {"dt": [30.0, 15.0], "maxC_error": e_t, "ratio": e_t
            [1] / e_t[0]}
277     add_check(report, "问题三_时间收敛", 0.35 < sf["ratio"] <
            0.65 and e_t[1] < e_t[0], sf, "误差递减, 收敛比接近0.5")
278
279     dr_list = [0.001, 0.0005, 0.00025]
280     sol = {dr: solve_problem3(dt=60.0, dr=dr, t_end=21600.0)
            for dr in dr_list}
281     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
282     final = {dr: np.interp(common_r, x[1], x[3][-1]) for dr, x
            in sol.items()}
283     e_s = [float(np.max(abs(final[0.001] - final[0.0005]))),
            float(np.max(abs(final[0.0005] - final[0.00025])))]
284     sf2 = {"dr": [0.0005, 0.00025], "maxC_error": e_s, "ratio":
            e_s[1] / e_s[0]}
285     add_check(report, "问题三_空间收敛", 0.15 < sf2["ratio"] <
            0.35 and e_s[1] < e_s[0], sf2, "误差递减, 收敛比接近0.25
            ")
286     return {"time": sf, "space": sf2}
287
288
289 def q3_full(report: dict):
290     # 复核问题三全过程
291     t, r, temp, moist, drying_time, max_c = solve_problem3()
292     stored = read_sheet(OUTPUT_DIR / "result3.xlsx").iloc[:,
            1:].to_numpy(dtype=float)
293     err = float(np.max(np.abs(np.round(moist[: stored.shape[0]
            - 1], 4) - stored[: -1])))
294     add_check(report, "问题三_独立重算", err <= 1e-12, err, "60
            s 网格重算与Excel一致")
295     j = int(np.flatnonzero(max_c <= MOISTURE_LIMIT)[0])
296     before, after = float(max_c[j - 1]), float(max_c[j])

```

```

297     frac = (drying_time - t[j - 1]) / (t[j] - t[j - 1])
298     c_end = moist[j - 1] + frac * (moist[j] - moist[j - 1])
299     monotonic = bool(np.all(np.diff(max_c) <= 1e-10))
300     physical = bool(np.all(np.isfinite(moist)) and np.all(moist
301                     >= -1e-12))
302     add_check(report, "问题三_阈值", before > MOISTURE_LIMIT
303             and after <= MOISTURE_LIMIT and abs(np.max(c_end) -
304             MOISTURE_LIMIT) < 1e-10,
305             {"drying_time_s": drying_time, "drying_time_h":
306             drying_time / 3600.0, "before": before, "after
307             ": after, "interpolated_max": float(np.max(
308             c_end))},
309             "首次跨越0.15并线性插值")
310     add_check(report, "问题三_单调与范围", monotonic and
311             physical,
312             {"monotonic": monotonic, "T_min": float(temp.min
313             ()), "T_max": float(temp.max()), "C_min":
314             float(moist.min()), "C_max": float(moist.max()
315             )}),
316             "max(C)单调不减; 有限值; C>=0")
317
318     v, area = control_volumes(0.001, len(r))
319     hm = 8e-7
320     t_func, c_func = __import__("q3").get_full_boundary_funcs()
321     c_air = np.asarray(c_func(t[1:]), dtype=float)
322     flux = hm * area * (c_air - moist[1:, -1])
323     storage = np.sum((moist[1:] - moist[:-1]) * v, axis=1)
324     m_res = abs(np.sum(storage) - np.sum(flux * 60.0)) / max(
325             abs(np.sum(storage)), 1e-12)
326     add_check(report, "问题三_守恒", m_res < 0.01, m_res, "水分
327             守恒相对残差<1%")
328     return {"t": t, "r": r, "T": temp, "C": moist, "maxC":
329             max_c, "drying_time": drying_time, "m_res": m_res}
330
331 def q4_recompute(report: dict):
332     # 重算问题四并检查守恒
333     q4_check = q4.validate_q4()
334     stored = read_sheet(OUTPUT_DIR / "result4.xlsx", 2).iloc[:,
335             1:-1].to_numpy(dtype=float)
336     t, radius, x, T, C, max_C, drying_time = q4.solve_problem4
337     ()

```

```

324     err = float(np.max(np.abs(np.round(C[:-1], 4) - stored
325         [:-1])))
326     add_check(report, "问题四_独立重算", err <= 1e-12, err, "无
        量纲网格结果与Excel四位小数一致")
327     add_check(report, "问题四_物理与守恒", q4_check["
        moisture_balance_abs"] < 1e-10,
        q4_check, "半径单调、恒温段最大水分单调、材料坐标
        积分恒等式成立")
328
329     short = {}
330     for dt in (120.0, 60.0, 30.0):
331         short[dt] = q4.solve_problem4(dt=dt, dx=0.05, t_end
            =18000.0, stop_at_threshold=False)
332     common_t = np.arange(0.0, 18000.0 + 1e-9, 120.0)
333     vals = {dt: s[4][np rint(common_t / dt).astype(int)] for dt
        , s in short.items()}
334     et = [float(np.max(abs(vals[120.0] - vals[60.0]))), float(
        np.max(abs(vals[60.0] - vals[30.0])))]
335     coarse = {}
336     for dx in (0.1, 0.05, 0.025):
337         coarse[dx] = q4.solve_problem4(dt=60.0, dx=dx, t_end
            =18000.0, stop_at_threshold=False)
338     ec = [float(np.max(abs(coarse[0.1][4][-1] - coarse
        [0.05][4][-1][::2]))),
339         float(np.max(abs(coarse[0.05][4][-1] - coarse
        [0.025][4][-1][::2])))]
340     conv = {"time_error": et, "time_ratio": et[1] / et[0], "
        space_error": ec, "space_ratio": ec[1] / ec[0]}
341     add_check(report, "问题四_步长收敛", et[1] < et[0] and ec
        [1] < ec[0]
342         and 0.35 < conv["time_ratio"] < 0.65 and 0.15 <
        conv["space_ratio"] < 0.35,
343         conv, "固定 xi 网格误差递减; 时间比约0.5, 空间比
        约0.25")
344     return {"check": q4_check, "convergence": conv}
345
346
347 def plot_convergence(report: dict):
348     PIC_DIR.mkdir(parents=True, exist_ok=True)
349     colors = {"T": "tab:red", "C": "tab:blue", "maxC": "tab:
        green"}
350     labels = [("问题一", report["convergence"]["q1"]), ("问题二

```

```

    ", report["convergence"]["q2"]), ("问题三", report["
convergence"]["q3"]))
351
352 fig, axes = plt.subplots(1, 3, figsize=(16, 4.8))
353 for ax, (title, data) in zip(axes, labels):
354     s = data["time"]
355     x = s["dt"]
356     for key in ["T_error", "C_error", "maxC_error"]:
357         if key in s:
358             ax.loglog(x, s[key], "o-", color=colors["T" if
key == "T_error" else "C" if key == "C_error
" else "maxC"], label=key.replace("_error",
""))
359     ax.set_title(f"{title} 时间步")
360     ax.set_xlabel("时间步长  $\Delta t$  (s)")
361     ax.set_ylabel("最大绝对误差")
362     ax.grid(alpha=0.3, which="both")
363     ax.legend()
364 fig.suptitle("三问时间步收敛交叉验证")
365 fig.tight_layout()
366 fig.savefig(PIC_DIR / "验证_时间步收敛_三问.png", dpi=300,
bbox_inches="tight")
367 plt.close(fig)
368
369 fig, axes = plt.subplots(1, 3, figsize=(16, 4.8))
370 for ax, (title, data) in zip(axes, labels):
371     s = data["space"]
372     x = s["dr"]
373     for key in ["T_error", "C_error", "maxC_error"]:
374         if key in s:
375             ax.loglog(x, s[key], "o-", color=colors["T" if
key == "T_error" else "C" if key == "C_error
" else "maxC"], label=key.replace("_error",
""))
376     ax.set_title(f"{title} 空间步")
377     ax.set_xlabel("空间步长  $\Delta r$  (m)")
378     ax.set_ylabel("最大绝对误差")
379     ax.grid(alpha=0.3, which="both")
380     ax.legend()
381 fig.suptitle("三问空间步收敛交叉验证")
382 fig.tight_layout()
383 fig.savefig(PIC_DIR / "验证_空间步收敛_三问.png", dpi=300,

```

```

        bbox_inches="tight")
384 plt.close(fig)
385
386
387 def plot_balance(q1: dict, q2: dict, q3: dict):
388     # 绘制守恒对照图
389     fig, axes = plt.subplots(2, 2, figsize=(14, 9))
390     # 问题一 能量和水分
391     v1, area1 = control_volumes(0.001, len(q1["r"]))
392     tf1, cf1 = get_q1_boundary()
393     at1, ac1 = np.asarray(tf1(q1["t"])), np.asarray(cf1(q1["t"]
        ]))
394     dq1 = np.cumsum(25.0 * area1 * (at1[1:] - q1["T"][:-1, -1])
        )
395     dm1 = np.cumsum(8e-7 * area1 * (ac1[1:] - q1["C"][:-1, -1])
        )
396     q_storage1 = np.cumsum(np.sum(820.0 * 2600.0 * np.diff(q1["
        T"], axis=0) * v1, axis=1))
397     m_storage1 = np.cumsum(np.sum(np.diff(q1["C"], axis=0) * v1
        , axis=1))
398     axes[0, 0].plot(q1["t"][1:], q_storage1, label="储能变化")
399     axes[0, 0].plot(q1["t"][1:], dq1, "--", label="边界热流积分
        ")
400     axes[0, 0].set_title(f"问题一能量守恒 (残差 {q1['q_res']:.2
        e}) ")
401     axes[0, 1].plot(q1["t"][1:], m_storage1, label="水分变化")
402     axes[0, 1].plot(q1["t"][1:], dm1, "--", label="边界质流积分
        ")
403     axes[0, 1].set_title(f"问题一水分守恒 (残差 {q1['m_res']:.2
        e}) ")
404
405     # 问题二和三 水分
406     v2, area2 = control_volumes(0.001, len(q2["r"]))
407     tf2, cf2 = get_q2_boundary(t_start=INITIAL_TIME)
408     ac2 = np.asarray(cf2(INITIAL_TIME + q2["t"]))
409     dm2 = np.cumsum(8e-7 * area2 * (ac2[1:] - q2["C"][1:, -1]))
410     ms2 = np.cumsum(np.sum(np.diff(q2["C"], axis=0) * v2, axis
        =1))
411     axes[1, 0].plot(q2["t"][1:] / 3600.0, ms2, label="水分变化"
        )
412     axes[1, 0].plot(q2["t"][1:] / 3600.0, dm2, "--", label="边
        界质流积分")

```

```

413 axes[1, 0].set_title(f"问题二水分守恒 (残差 {q2['m_res']:.2e}) ")
414
415 v3, area3 = control_volumes(0.001, len(q3["r"]))
416 from q3 import get_full_boundary_funcs
417
418 _, cf3 = get_full_boundary_funcs()
419 ac3 = np.asarray(cf3(q3["t"]))
420 dm3 = np.cumsum(8e-7 * area3 * (ac3[1:] - q3["C"][1:, -1])
421                  * 60.0)
422 ms3 = np.cumsum(np.sum(np.diff(q3["C"], axis=0) * v3, axis
423                          =1))
424 axes[1, 1].plot(q3["t"][1:] / 3600.0, ms3, label="水分变化")
425 axes[1, 1].plot(q3["t"][1:] / 3600.0, dm3, "--", label="边界质流积分")
426 axes[1, 1].set_title(f"问题三水分守恒 (残差 {q3['m_res']:.2e}) ")
427
428 for ax in axes.flat:
429     ax.set_xlabel("时间 (s)" if ax in axes[0, :] else "时间(h)")
430     ax.set_ylabel("单位长度储量变化")
431     ax.grid(alpha=0.3)
432     ax.legend()
433 fig.suptitle("守恒定律交叉验证: 储量变化与边界通量积分")
434 fig.tight_layout()
435 fig.savefig(PIC_DIR / "验证_守恒对照_三问.png", dpi=300,
436             bbox_inches="tight")
437 plt.close(fig)
438
439 def plot_cross_check(report: dict):
440     names = ["Q1重算T", "Q1重算C", "Q2重算T", "Q2重算C", "5500s断点T", "5500s断点C"]
441     vals = [
442         report["checks"]["问题一_独立重算"]["value"]["T_max_abs"],
443         report["checks"]["问题一_独立重算"]["value"]["C_max_abs"],
444         report["checks"]["问题二_独立重算"]["value"]["T_max_abs"],
445     ],

```

```

443     report["checks"]["问题二_独立重算"]["value"]["C_max_abs
        "],
444     report["checkpoint"]["T_max_abs"], report["checkpoint"
        ]["C_max_abs"],
445 ]
446 vals_plot = np.maximum(vals, 1e-16)
447 fig, ax = plt.subplots(figsize=(10, 5))
448 bars = ax.bar(names, vals_plot, color=["tab:red", "tab:blue
        "] * 3)
449 ax.set_yscale("log")
450 ax.set_ylabel("最大绝对差异（对数坐标）")
451 ax.set_title("独立重算与5500 s断点交叉校验")
452 ax.grid(axis="y", alpha=0.3, which="both")
453 for bar, value in zip(bars, vals):
454     ax.text(bar.get_x() + bar.get_width() / 2, bar.
        get_height(), f"{value:.2e}", ha="center", va="
        bottom", fontsize=9)
455 fig.tight_layout()
456 fig.savefig(PIC_DIR / "验证_独立重算交叉校验.png", dpi=300,
        bbox_inches="tight")
457 plt.close(fig)
458
459
460 def plot_q4_validation(report: dict):
461     # 绘制问题四检验图
462     conv = report["checks"]["问题四_步长收敛"]["value"]
463     fig, axes = plt.subplots(1, 2, figsize=(10, 4))
464     axes[0].loglog([120, 60], conv["time_error"], "o-")
465     axes[0].set_xlabel("时间步长/s")
466     axes[0].set_ylabel("最大误差")
467     axes[0].set_title(f"时间步收敛比 {conv['time_ratio']:.4f}")
468     axes[0].grid(alpha=0.3, which="both")
469     axes[1].loglog([0.1, 0.05], conv["space_error"], "o-",
        color="tab:orange")
470     axes[1].set_xlabel("无量纲空间步长")
471     axes[1].set_ylabel("最大误差")
472     axes[1].set_title(f"空间步收敛比 {conv['space_ratio']:.4f}"
        )
473     axes[1].grid(alpha=0.3, which="both")
474     fig.suptitle("问题四移动边界步长收敛")
475     fig.tight_layout()
476     fig.savefig(PIC_DIR / "验证_问题四收敛.png", dpi=300,

```

```

        bbox_inches="tight")
477 plt.close(fig)
478
479
480 def main():
481     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
482     PIC_DIR.mkdir(parents=True, exist_ok=True)
483     report = {"checks": {}, "convergence": {}}
484     validate_file_structure(report)
485     q1 = q1_recompute_and_balance(report)
486     q2 = q2_recompute_and_balance(report)
487     q3 = q3_full(report)
488     q4_result = q4_recompute(report)
489
490     # 比较5500 s断点
491     t_func, c_func = get_q2_boundary(path=OUTPUT_DIR / "output1
        .xlsx", t_start=0.0)
492     _, r_chk, t_chk, c_chk = solve_q2(
493         1.0, 0.001, t_end=5500.0, t_start=0.0, initial_file=
        None,
494         initial_T=28.0, initial_C=2.55, T_air_func=t_func,
        C_air_func=c_func,
495     )
496     _, t_ref, c_ref = load_initial_profile(DEFAULT_INITIAL_FILE
        , r_max=R_MAX, Nr=len(r_chk))
497     checkpoint = {"T_max_abs": float(np.max(abs(t_chk[-1] -
        t_ref))), "C_max_abs": float(np.max(abs(c_chk[-1] -
        c_ref)))}
498     add_check(report, "问题二_5500s断点", checkpoint["T_max_abs
        "] <= 1e-4 and checkpoint["C_max_abs"] <= 1e-4,
        checkpoint, "全过程重算与output3最大差异<=1e-4")
499     report["checkpoint"] = checkpoint
500
501     report["convergence"]["q1"] = convergence_q1(report)
502     report["convergence"]["q2"] = convergence_q2(report)
503     report["convergence"]["q3"] = convergence_q3(report)
504     report["convergence"]["q4"] = q4_result["convergence"]
505     plot_convergence(report)
506     plot_balance(q1, q2, q3)
507     plot_cross_check(report)
508     plot_q4_validation(report)
509

```

```

510     report["summary"] = {
511         "all_passed": True,
512         "q3_drying_time_s": q3["drying_time"],
513         "q3_drying_time_h": q3["drying_time"] / 3600.0,
514         "q4_drying_time_s": q4_result["check"]["drying_time_s"]
515             ],
516         "q4_drying_time_h": q4_result["check"]["drying_time_s"]
517             / 3600.0,
518         "q4_radius_start_cm": q4_result["check"]["radius_start_cm"],
519         "q4_radius_end_cm": q4_result["check"]["radius_end_cm"]
520             ],
521         "figure_files": [
522             "q1_综合图.png",
523             "q1_三维图.png",
524             "q2_综合图.png",
525             "q2_三维图.png",
526             "q3_综合图.png",
527             "q3_三维图.png",
528             "q4_综合图.png",
529             "q4_三维图.png",
530             "验证_时间步收敛_三问.png",
531             "验证_空间步收敛_三问.png",
532             "验证_守恒对照_三问.png",
533             "验证_独立重算交叉校验.png",
534             "验证_问题四收敛.png",
535         ],
536     }
537     REPORT_FILE.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
538     print(f"全部交叉验证通过, 报告已保存: {REPORT_FILE}")
539     print(json.dumps(report["summary"], ensure_ascii=False, indent=2))
540
541 if __name__ == "__main__":
542     main()

```